

“EVENT MANAGEMENT AND ANALYTICS PORTAL”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

B. HEMA SRI :2520030159

P. HAVISHAA REDDY: 2520030371

K. BHARGAVI :2520030074

Under the Esteemed Guidance of

Dr. Spandana

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled **“EVENT MANAGEMENT AND ANALYTICS PORTAL”** is a record of Bonafide work of **B. HEMA SRI-2520030159, P. HAVISHAA REDDY-2520030371, K. BHARGAVI -2520030074** submitted in

partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

B. HEMA SRI :2520030159

P. HAVISHAA REDDY: 2520030371

K. BHARGAVI :2520030074

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **“EVENT MANAGEMENT AND ANALYTICS PORTAL”** is a Bonafide work done and submitted by **B. HEMA SRI – 2520030159 , P. HAVISHAA REDDY – 2520030371 , K. BHARGAVI - 2520030074** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Ramatenki Sateesh Kumar**, and my guide **Dr Spandana**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)** Head of the *Department for Computer Science Engineering*/ for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

B. HEMA SRI :2520030159

P. HAVISHAA REDDY: 2520030371

K. BHARGAVI :2520030074

Abstract

The **Event Management and Analytics Portal** is a web-based application designed to simplify and organize the process of creating, managing, registering for, and monitoring events. The system provides different functionalities for organizers, staff, and attendees through a role-based interface.

The portal allows organizers to create and manage events by providing details such as event name, category, date, venue, and capacity. Attendees can view available events, search and filter events, and register for events. Staff and organizers can perform attendee check-in using QR codes or manual ticket codes.

The system also provides an audit log module to record important activities and an analytics dashboard to display event-related information using charts and visual representations. Authentication and role-based access control are used to restrict certain operations to authorized users.

The project is developed using a React-based frontend with supporting API integration through Axios. The proposed backend architecture uses Node.js and Express.js, with MySQL planned as the database layer. The current project implementation primarily demonstrates the frontend functionality, client-side authentication, event management workflow, QR check-in, audit logs, and analytics using sample data.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5-15
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	16-21
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	22-32
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	33-36
5	Design	5.2 ER Diagram / Database schema	37
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	38-47
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	48-55
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	56-64
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	65-73
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	74-82
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	83-88
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	89-97
13	References	- Books, tutorials, APIs, documentation sites used	98-99

14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	100-105
----	------------	--	---------

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Events play an important role in educational institutions, organizations, companies, and other communities. Conferences, workshops, seminars, hackathons, technical competitions, cultural programs, training sessions, and management events are conducted regularly to provide opportunities for learning, collaboration, skill development, and interaction. As the number of events increases, managing the complete event lifecycle becomes more complex. Event organizers need to maintain information about events, participants, registrations, attendance, and other activities in an organized manner.

In a traditional event management process, different activities are often handled separately. Event details may be maintained using spreadsheets or documents, registration information may be collected through forms, and attendance may be recorded manually. Organizers may also need to maintain separate records for participants and event activities. When the number of participants increases, manually maintaining these records can become difficult and time-consuming. It can also make it harder to obtain a clear overview of event participation and attendance.

The **Event Management and Analytics Portal** is proposed as a web-based application to provide a centralized platform for managing important event-related activities. Instead of depending on separate methods for different tasks, the portal brings multiple activities into a single application. The system is designed to provide different functionalities for organizers, staff, and attendees based on their roles.

One of the important aspects of the project is event management. Organizers need to create events by providing information such as event name, category, date, venue, and capacity. A centralized event management interface makes this information easier to maintain and present to users. Attendees can view the available events and obtain the required information before deciding to participate.

The portal also focuses on improving the event discovery process. When multiple events are available, users may need to find a particular event quickly. Search and filtering functionality helps users locate relevant events without manually going through every available event. This provides a more organized way of presenting event information and improves the overall usability of the system.

Another important part of event management is participant registration. After discovering an event, an attendee should be able to register through the same platform. Integrating event information and registration into one system reduces the need for separate registration processes. It also creates a structured workflow from event discovery to participation.

Attendance management is another major requirement of an event management system. Traditional attendance methods may involve manually checking participant information, which can take considerable time when many attendees are present. The proposed portal includes a QR-based check-in functionality. Authorized users such as organizers and staff can use the check-in module to verify attendees using QR codes. A manual ticket-code option is also provided as an alternative when QR scanning cannot be used.

The system also includes an **audit log** functionality. During event management, several important activities can take place, such as creating an event or checking in an attendee. Maintaining records of such activities provides a way to monitor actions performed within the application. Audit information can therefore support better monitoring and accountability of event-related operations.

The project also introduces an **analytics dashboard** for presenting event-related information in a visual form. Event management does not only involve conducting an event; organizers may also need to understand participation and attendance information. Presenting information through charts can make numerical data easier to understand. The analytics dashboard provides a visual interface for displaying event-related statistics and can serve as a foundation for more advanced reporting in future versions of the system.

Role-based access is another important concept used in the project. Different users have different responsibilities in an event management environment. An organizer may need access to event creation, analytics, and audit logs, while staff members may primarily require access to attendee check-in. Attendees mainly need functionality

related to viewing and registering for events. Providing access according to user roles helps organize the system and prevents every user from accessing all available operations.

The project is developed around a web-based architecture with a React frontend. React provides a component-based structure for developing the user interface. The application contains separate pages and reusable components for authentication, event management, check-in, analytics, and audit logs. An API service using Axios is also included to provide a structured mechanism for communication between the frontend and a backend service.

The proposed architecture considers Node.js and Express.js as the backend technologies and MySQL as the database technology. These technologies can be used to provide server-side processing and persistent storage for users, events, registrations, attendance records, and audit information. The submitted project primarily demonstrates the frontend application and client-side workflows, while the backend and database components form the planned integration layer for a complete deployment.

The development of this project is therefore focused on creating a structured and user-friendly event management environment. By bringing event creation, event discovery, registration, check-in, audit logging, and analytics into one portal, the system provides a common platform for different users involved in an event.

The project also provides a foundation that can be extended in future versions. A complete backend and database implementation can provide persistent storage and server-side processing. Additional capabilities such as email notifications, event reminders, advanced reports, capacity management, waitlists, and real-time analytics can also be incorporated later.

Overall, the **Event Management and Analytics Portal** is designed to demonstrate how a web-based application can organize the major activities involved in event management. The project combines a structured user interface, role-based access, event management features, QR-based attendance verification, audit logging, and analytics into a single platform. This approach provides a foundation for improving the organization and monitoring of events while also allowing the system to be expanded with additional backend, database, security, and reporting capabilities.

1.2 Problem Statement

Managing events involves several activities, including creating events, maintaining event information, registering participants, verifying attendance, monitoring activities, and analyzing participation. When these activities are handled through separate or manual processes, event organizers may face difficulties in maintaining accurate and up-to-date information. The absence of a centralized system can make event management less organized, especially when multiple events and a large number of participants are involved.

In a conventional event management process, organizers may maintain event details using documents, spreadsheets, forms, or other separate tools. Participant registration may be handled independently from event information, while attendance may be recorded manually during the event. As a result, information related to an event can be distributed across different sources. This makes it difficult for organizers to obtain a complete view of an event and its participants from a single location.

One of the major problems is the **management of event information**. Organizers need to maintain details such as event name, category, date, venue, and participant capacity. When event information is maintained manually, updating or accessing the latest information can become inconvenient. There is also a need for a simple interface through which organizers can create events and users can view available events.

Another problem is **event discovery**. When an organization conducts multiple events, attendees may find it difficult to identify a particular event from a large list of available programs. Without proper search and filtering mechanisms, users may have to manually examine each event. This can make the process of finding relevant events less convenient.

Participant registration is another important issue. If registration is managed separately from the event management process, organizers may have to maintain participant information independently.

This can increase the effort required to manage registrations and may make it difficult to connect participant information with the corresponding event.

Attendance management also presents a significant challenge. During an event, organizers or staff members need to verify whether registered participants have actually arrived. Manual attendance verification can take considerable time, particularly when many participants are present. It may also require additional effort to maintain attendance records after the event. Therefore, there is a need for a faster and more organized check-in mechanism.

To address this requirement, the proposed system provides a **QR-based check-in mechanism**.

Authorized organizers and staff can scan an attendee's QR code to perform the check-in. A manual ticket-code option is also provided as an alternative. This creates a structured attendance verification process within the same event management portal.

Another problem is the lack of **centralized monitoring of system activities**. Important operations such as event creation and attendee check-in need to be monitored by organizers. If there is no record of these activities, it becomes difficult to understand what actions have been performed within the system. The proposed audit-log module addresses this requirement by providing a place to record and view important activities.

Event organizers may also require information about participation and attendance to understand how events are performing. When event information is stored without suitable visualization, analyzing the available data can require additional manual effort. Therefore, the system includes an **analytics dashboard** that presents event-related information through graphical representations. This provides a foundation for understanding event participation and attendance information.

Another important issue is **access control**. Not every user should have access to every operation in an event management system. For example, an organizer may need to create events and access analytics and audit logs, while staff members may require access to check-in functionality. Attendees mainly require access to event information and registration. Without appropriate role-based access, users may be provided with unnecessary functionality or access to operations that are not relevant to their responsibilities.

The proposed **Event Management and Analytics Portal** addresses these problems by providing a centralized web-based environment for event-related activities. The system combines event management, event discovery, registration, check-in, audit logging, notifications, and analytics within a single application. React is used for the frontend interface, while an API integration layer is provided for communication with the planned backend.

The project also considers a Node.js and Express.js backend with MySQL as the database layer. These components can be used to provide persistent storage and server-side processing in a complete implementation. The submitted project primarily demonstrates the frontend and client-side workflows, while backend and database integration can be implemented as part of further development.

Therefore, the main problem addressed by this project is the need for a **centralized, organized, and user-friendly system for managing event-related activities**. The proposed portal aims to reduce dependence on disconnected processes by bringing important event operations into one platform and providing role-specific functionality for organizers, staff, and attendees.

The problem can be summarized through the following key areas:

1. **Lack of centralized event information** – Event details may be maintained separately, making them difficult to manage and access.
2. **Difficulty in finding events** – Users may need search and filtering facilities to quickly identify relevant events.
3. **Separate registration processes** – Participant registration needs to be connected with event information.
4. **Manual attendance verification** – Traditional attendance methods can be time-consuming, especially for large events.
5. **Limited activity monitoring** – Important actions require an organized audit-log mechanism.
6. **Difficulty in analyzing event information** – Organizers need visual representations to understand participation and attendance data.
7. **Lack of role-based access** – Different users require different levels of access depending on their responsibilities.

8. **Need for an extensible architecture** – The system should provide a foundation for future backend, database, security, reporting, and analytics enhancements.

The Event Management and Analytics Portal is therefore designed to address these requirements through a single web-based application that integrates the major activities involved in event management.

1.3 Scope of the Project

The scope of the **Event Management and Analytics Portal** covers the development of a centralized web-based application for managing the major activities involved in organizing and participating in events. The system is designed to provide a common platform where organizers, staff, and attendees can perform their respective activities through a structured and user-friendly interface. The project focuses on reducing the dependency on separate or manual methods for managing event information, registrations, attendance, activity tracking, and event-related analytics.

The primary scope of the project is **event management**. The system provides functionality for authorized organizers to create events by entering important information such as event name, event category, event date, venue, and participant capacity. The created events can then be displayed through the event listing interface, allowing users to access the available event information. This provides a centralized method of presenting event details and creates a basic workflow for managing events within the application.

The project also covers **event discovery and searching**. When multiple events are available, users need a convenient way to find the event they are interested in. The portal provides an event listing along with search and filtering functionality so that users can locate relevant events more easily. This functionality is intended to improve the user experience by reducing the effort required to manually go through all available events.

Another important area within the scope of the project is **user registration and authentication**. Users can register and log in to the application before accessing the features available to them. The application maintains authentication information through an authentication context and uses protected routes to control access to certain pages. This provides the foundation for managing different categories of users within the application.

The system also includes **role-based access control**. Different users have different responsibilities in an event management environment, and therefore they should not necessarily have access to every functionality. Organizers are provided with access to functions such as event creation, analytics, audit logs, and check-in. Staff members can access the check-in functionality, while attendees can view available events and participate through the registration process. The role-based structure helps organize the application according to the responsibilities associated with each type of user.

The scope of the project includes **attendee event registration**. After viewing an event and its details, an attendee can register for the event through the portal. This creates a connection between event discovery and participation. The registration functionality is intended to provide a simple and organized method for attendees to participate in available events instead of depending on separate registration processes.

Attendance management and check-in are also important parts of the project scope. The system provides a QR-based check-in module that allows authorized users such as organizers and staff to verify attendees using QR codes. The project uses QR scanning functionality to support this process. A manual ticket-code option is also provided so that an attendee can be verified even when QR scanning is not possible. This functionality is intended to make the attendance verification process more organized and convenient during an event.

The project also covers **audit logging** for important activities performed within the application. The audit-log module provides a way to record and display activities such as event creation and attendee check-in. Maintaining these records can help organizers monitor important actions performed within the system. The audit-log functionality also provides a foundation for maintaining better accountability and activity tracking in a complete event management environment.

Another major area within the scope is the **analytics dashboard**. The system provides a dashboard that uses graphical representations to display event-related information. Analytics can help organizers understand participation and attendance information in a more visual and understandable manner. The project uses chart-based visualization to demonstrate this functionality. The current

implementation uses client-side or sample information, while a future database-backed implementation can provide dynamic analytics based on actual registration and attendance records. The scope of the project also includes **notifications**. A notification banner is provided to display messages and feedback to users when certain operations are performed. This allows the system to communicate information to users through the interface and improves interaction between the user and the application.

From a technical perspective, the project includes an **API integration layer** using Axios. This layer provides a structured mechanism through which the React frontend can communicate with a backend service. The API service is designed to use a configurable backend URL and can attach authentication information to requests. This provides a foundation for integrating the frontend with a server-side application in the future.

The proposed system architecture includes **Node.js and Express.js** as the backend technologies and **MySQL** as the database layer. These technologies can be used to provide server-side processing and persistent storage for users, events, registrations, attendance records, and audit logs. However, the submitted project source primarily contains the React frontend and client-side/sample data. A complete Express backend and MySQL database implementation are not included in the submitted project source. Therefore, backend and database integration are considered part of the planned architecture and future expansion of the project rather than fully implemented functionality in the current version.

The scope of the project can be further extended after completing the backend and database integration. Persistent storage can be introduced so that user, event, registration, attendance, and audit information can be maintained even after the application is closed. Server-side authentication and authorization can also be implemented to provide stronger access control. Additional functionality such as event editing, event cancellation, capacity management, waitlists, email notifications, event reminders, detailed reports, and real-time analytics can be added in future versions.

The project is mainly focused on **event management, event discovery, participant registration, attendance verification, activity monitoring, and analytics**. It does not currently focus on unrelated event-management areas such as transportation management, accommodation booking, financial accounting, or complete enterprise-level event operations. Keeping the project within this defined scope allows the major event management workflows to be demonstrated clearly while providing a foundation for further development.

Overall, the scope of the Event Management and Analytics Portal is to provide a centralized and structured platform for handling the major activities associated with events. By combining event creation, event listing, searching, registration, role-based access, QR-based check-in, audit logging, notifications, and analytics within one application, the project provides a foundation for a more organized event management process. The architecture also allows the system to be expanded with a complete backend, database, enhanced security, persistent storage, and advanced reporting capabilities in future development.

1.4 Objectives of the Project

The primary objective of the **Event Management and Analytics Portal** is to develop a centralized web-based platform that simplifies and organizes the major activities involved in event management. The system is designed to bring event creation, event discovery, participant registration, attendance verification, activity monitoring, and analytics into a single application. By providing these functionalities through one platform, the project aims to provide a structured environment for organizers, staff, and attendees to interact with event-related information.

One of the main objectives of the project is to **simplify event creation and management**. Event organizers need to maintain important information such as the event name, category, date, venue, and participant capacity. The portal provides an interface through which authorized organizers can enter and manage this information. This objective is intended to provide a more organized method of maintaining event details and presenting them to users.

Another objective is to **provide an easy method for users to discover available events**. When several events are conducted, users should be able to find the required event without going through every event manually. The system therefore provides event listing, search, and filtering functionality. This helps users locate relevant events and access important event information through a common interface.

The project also aims to **simplify the participant registration process**. Attendees should be able to view an event and register for it through the same application. Integrating event information and registration into one platform creates a simple workflow for users and reduces the need to use separate systems for discovering and registering for events.

An important objective of the project is to **provide an organized attendance verification mechanism**. The system includes a QR-based check-in facility that allows authorized organizers and staff to verify attendees using QR codes. A manual ticket-code option is also provided as an alternative method. This objective focuses on making the check-in process more structured and convenient during events.

Another objective is to **implement role-based access within the application**. Organizers, staff, and attendees have different responsibilities and therefore require access to different functionalities. The system is designed to provide organizers with functions such as event creation, analytics, audit logs, and check-in, while staff members can perform check-in operations and attendees can view and register for events. This objective helps organize system access according to user responsibilities.

The project also aims to **maintain records of important system activities** through an audit-log mechanism. Activities such as event creation and attendee check-in can be recorded and displayed through the audit-log module. The objective is to provide organizers with better visibility into important operations performed within the application and to establish a foundation for activity monitoring.

Another important objective is to **provide event-related analytics through visual representations**. The analytics dashboard is designed to present event information using charts and graphical elements. Instead of relying only on raw numerical information, the dashboard provides a visual way of understanding participation and attendance-related information. This can help organizers interpret available event information more easily.

The project also aims to **improve communication between the application and its users** through notification functionality. Notification messages can be displayed within the application to provide feedback about different operations. This helps users understand whether an action has been completed or whether additional information needs to be considered.

From a technical perspective, an objective of the project is to **develop a structured and reusable frontend architecture using React**. The application is divided into different pages, components, services, and authentication-related modules. This component-based structure makes the application easier to organize and provides a foundation for extending the system with additional functionality.

Another objective is to **provide an API integration structure for future backend communication**. The project uses Axios as an API communication layer, allowing the React frontend to communicate with a backend service through configured API endpoints. The structure also supports sending authentication information with API requests. This creates a foundation for connecting the application to a complete server-side implementation.

The proposed architecture also aims to support **persistent data management through a backend and database**. Node.js and Express.js are considered for backend development, while MySQL is considered for storing user, event, registration, attendance, and audit information. Although the current submitted project primarily demonstrates the frontend and client-side workflow, the architecture is designed so that these components can be integrated in future development.

The project also aims to provide a **user-friendly and organized interface**. Since the portal is intended for different types of users, the interface should present relevant information clearly and provide straightforward navigation between different functionalities. Reusable components such as navigation elements, protected routes, notification banners, forms, tables, and dashboards help maintain consistency throughout the application.

Finally, the overall objective of the project is to establish a **foundation for a complete event management and analytics system** that can be expanded in the future. The system can later support persistent database storage, secure server-side authentication, advanced event management, real-time analytics, automated notifications, detailed reports, capacity management, and other features. Therefore, the project is intended not only to demonstrate the major event management workflows but also to provide an extensible structure that can support further development.

In summary, the objectives of the Event Management and Analytics Portal focus on creating a centralized platform for managing events, simplifying event discovery and registration, providing QR-based attendance verification, controlling access according to user roles, monitoring important activities, and presenting event information through analytics. These objectives collectively support the development of an organized and user-friendly system for managing event-related activities.

1.5 Importance of the Application

The **Event Management and Analytics Portal** is important because it provides a centralized platform for organizing and managing different activities associated with events. In many event management environments, information related to event details, participant registration, attendance, and other activities may be handled separately. This can make the overall process more difficult to manage, especially when several events and a large number of participants are involved. The proposed application brings these major activities together within a single web-based platform.

One of the major advantages of the application is that it **simplifies event management for organizers**. Organizers can create events by entering important information such as event name, category, date, venue, and capacity. Having a dedicated interface for managing this information provides a more structured way of handling events. It also allows event information to be presented consistently to users through the event listing interface.

The application is also important from the perspective of **event discovery and accessibility**. Attendees can view the available events and use search and filtering functionality to find events that are relevant to them. Instead of depending on separate sources to obtain event information, users can access the available event details through the portal. This provides a more convenient interaction between attendees and the event management system.

Another important aspect is **participant registration**. The application provides a registration workflow through which attendees can register for events after viewing their details. Integrating registration with the event management platform creates a connected process from discovering an event to participating in it. This can make event participation more organized and provide a foundation for maintaining registration information.

The application is particularly important for **attendance management** because it includes QR-based check-in functionality. During an event, verifying attendees can become time-consuming when it is performed manually. The QR check-in module allows authorized organizers or staff members to scan attendee QR codes. The application also provides a manual ticket-code option, giving users an alternative method of verification. This makes the check-in process more flexible and structured.

The application also provides importance through **role-based access control**. Organizers, staff, and attendees have different responsibilities within an event environment. The portal provides different access to functionalities according to these roles. Organizers can access features such as event creation, analytics, and audit logs, while staff can perform check-in operations and attendees can access event discovery and registration functionality. This separation helps organize the system according to the responsibilities of different users.

Another important feature is the **audit-log functionality**. Events may involve multiple activities performed by different users, and maintaining records of important actions can help organizers monitor the system. The audit-log module provides a structured location for displaying activities such as event creation and attendee check-in. This can improve visibility into the operations performed through the application.

The **analytics dashboard** is also an important part of the application. Event organizers may need to understand information related to registrations and attendance rather than simply viewing individual records. Presenting event-related information through charts provides a more visual method of understanding the available data. The analytics functionality therefore provides a foundation for organizers to examine event-related information and can be extended in the future with database-driven analytics.

The application is also important from a **usability and organization perspective**. Instead of requiring users to interact with multiple independent systems, the portal provides several event-related functionalities through a common interface. The React-based interface is organized into separate pages and reusable components, making it easier for users to navigate between the available functions.

From a technical perspective, the application demonstrates the use of **modern web development concepts**. The project uses React for the frontend, authentication context for managing authentication state, protected routes for restricting access, Axios for API communication, QR-code scanning functionality for check-in, and chart-based visualization for analytics. These components demonstrate how different technologies can be combined to develop a functional event management application.

The application also provides a **foundation for future backend and database integration**. The project contains an API service that can communicate with a backend application. The proposed architecture considers Node.js and Express.js for server-side processing and MySQL for persistent data storage. Although the submitted project primarily demonstrates the frontend and client-side workflow, this structure allows the application to be extended into a complete client-server system.

Another important aspect of the application is its **potential for scalability and future enhancement**. As the requirements of an event management system increase, additional features can be integrated into the existing structure. Persistent user and event data, secure server-side authentication, email notifications, event reminders, capacity management, waitlists, detailed reports, and real-time analytics can be added in future versions. The existing modular structure provides a foundation for such improvements.

The application is also useful as an **academic project because it demonstrates the practical application of software development concepts**. It combines frontend development, component-based design, authentication, role-based access, API integration, QR-code processing, data visualization, and system organization within one project. This demonstrates how different concepts can be integrated to solve a practical problem.

Overall, the importance of the **Event Management and Analytics Portal** lies in its ability to bring major event-related activities into a centralized application. It provides a structured environment for organizers to manage events, attendees to discover and register for events, and authorized staff to verify attendance. At the same time, audit logs and analytics provide additional functionality for monitoring and understanding event activities. The project therefore provides a foundation for developing a more

complete and scalable event management solution through future backend, database, security, and analytics enhancements.

1.6 Target Users / Audience

The **Event Management and Analytics Portal** is primarily designed for event organizers who are responsible for creating and managing events. Organizers can add event details such as event name, category, date, venue, and capacity, and can access features such as event management, analytics, audit logs, and attendee check-in. The system provides them with a centralized platform to monitor and manage important event-related activities.

The application is also intended for **event staff and attendees**. Staff members can use the check-in functionality to verify attendees using QR codes or manual ticket codes. Attendees can use the portal to view available events, search and filter events, and register for events. The role-based structure ensures that users can access the functions relevant to their responsibilities.

The portal can be used by **educational institutions, organizations, companies, and other groups** that conduct events such as conferences, workshops, seminars, hackathons, and training programs. By providing event management, registration, attendance verification, audit logging, and analytics in one application, the system provides a common platform for different users involved in the event lifecycle.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The **Event Management and Analytics Portal** is a web-based application and does not require specialized hardware for normal operation. The system can be developed, tested, and accessed using standard computer systems available in an academic institution or office environment. The hardware requirements mainly depend on whether the system is being used by an administrator or organizer for development and management, or by an attendee for accessing the portal through a web browser.

For development and administrative usage, a computer or laptop with a modern processor such as an **Intel Core i3 or equivalent and above** is sufficient. A minimum of **4 GB RAM** is recommended for basic operation, while **8 GB RAM or more** is preferable when running the development environment along with the browser and other supporting applications. The system should have at least **10 GB of available storage** for the project source code, development tools, dependencies, and temporary files. An **SSD** is recommended because it provides faster application loading, dependency installation, and development operations compared with a traditional hard disk.

The system also requires a **display, keyboard, and pointing device** such as a mouse or touchpad for interacting with the portal and development environment. A display with a resolution of at least **1366 × 768 pixels** is adequate for using the application, although a higher-resolution display can provide a better experience when viewing dashboards, tables, and analytics charts. For the QR-based check-in functionality, a **device with a camera**, such as a smartphone, tablet, laptop, or webcam-equipped computer, may be required because the system uses QR scanning for attendee check-in. The camera allows event staff to scan the QR code associated with an attendee's registration.

For users accessing the portal, a standard desktop computer, laptop, tablet, or smartphone with a modern web browser can be used. The device should have sufficient processing capability to load the web interface, display event information, complete registration, and interact with the dashboard. A stable **internet or local network connection** is also required when the application communicates with external or backend services through the API layer. In the current project implementation, the frontend can operate using the available client-side application data, while the planned backend and database integration will require network connectivity between the client application and server.

For server-side deployment in the future, the application can be hosted on a computer or cloud-based server with adequate CPU, RAM, storage, and network connectivity. The exact server configuration will depend on the number of users, number of events, amount of stored registration data, and expected frequency of access. Since the current submitted project primarily contains the frontend implementation and the backend/database are planned for integration, a specific production-server hardware configuration is not fixed at this stage. The hardware can therefore be scaled according to the requirements of the institution or organization deploying the **Event Management and Analytics Portal**.

2.2 Software Requirements

The Event Management and Analytics Portal is developed as a web-based application using modern web development technologies. The software environment consists of the operating system, web browser, programming environment, frontend libraries, API communication tools, and supporting development packages required to develop and execute the application. The following software requirements provide the basic environment necessary for developing, testing, and using the system.

The application requires a modern Windows, Linux, or macOS operating system. During development, the system can be executed on a standard development computer using an appropriate operating system with support for Node.js and modern web browsers. The operating system is responsible for providing the environment required to install dependencies, execute development commands, manage project files, and run the application locally.

The project uses Node.js and npm (Node Package Manager) as the development environment for the React-based frontend. Node.js provides the runtime environment required to execute the development tools, while npm is used to install and manage the packages required by the application. The project uses React.js for developing the frontend interface. React provides a component-based structure for creating pages such as login, registration, event listing, event creation, QR check-in, audit logs, and the analytics dashboard.

A modern web browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox is required to access and test the portal. The browser should support modern JavaScript, HTML5, CSS, and browser APIs used by the application. The QR check-in functionality also requires browser support for accessing the device camera when QR scanning is performed. Browser developer tools can additionally be used during development for checking errors, inspecting components, and debugging network requests.

The project also uses Axios for communication between the frontend application and the API layer. The Axios service is configured to communicate with an API endpoint through an environment variable such as `REACT_APP_API_URL`, with a local API address available as the default configuration. Authentication information can be attached to API requests through a Bearer token mechanism. The submitted implementation contains the frontend API integration layer, while the complete server-side API is planned for integration with the backend.

The application uses additional frontend libraries to support its functionality. `html5-qrcode` is used for QR-code scanning during event check-in, while `Chart.js` and `react-chartjs-2` are used to display analytics and graphical information in the analytics dashboard. CSS is used for the visual design and responsive layout of the application, including cards, forms, tables, buttons, badges, and other interface components.

For development and maintenance, a code editor such as Visual Studio Code can be used to edit the project files and manage the source code. The project can also be executed using the terminal or command prompt through standard npm commands. Browser developer tools and console output can be used to identify frontend errors and verify application behavior during testing.

The project presentation describes Node.js/Express and MySQL as the intended backend and database technologies. These components are planned for providing server-side processing, persistent event and registration data, authentication, and database management. However, the submitted project source primarily contains the React frontend implementation; therefore, the complete Express backend and MySQL database are not considered implemented components of the current source version. They can be integrated as part of the next development stage.

Overall, the software environment provides the necessary foundation for developing, running, testing, and extending the Event Management and Analytics Portal. The combination of React, Node.js/npm, Axios, QR-scanning support, Chart.js, and modern web browsers allows the application to provide its major frontend features while maintaining an architecture that can later be connected to a persistent backend and database.

2.3 Development Tools and Frameworks

The **Event Management and Analytics Portal** is developed using a set of modern web development tools and frameworks that support the creation, testing, debugging, and maintenance of the application. These tools provide the necessary environment for developing the frontend interface, managing project dependencies, communicating with APIs, implementing QR-based check-in, and displaying analytics. The major development tools and frameworks used or planned for the project are described below.

React.js is the primary frontend framework used for developing the portal. React follows a component-based approach, which allows the application interface to be divided into reusable components and pages. In the project, React is used to create important interfaces such as the login page, registration page, event listing, event creation, QR check-in, audit logs, and analytics dashboard. The use of components makes the application easier to organize and maintain because individual parts of the interface can be developed and modified separately.

Node.js and npm provide the development environment and package management support for the project. Node.js is used to run the development environment, while npm is used to install, manage, and maintain the packages required by the React application. The project dependencies are maintained through the package configuration, allowing the required libraries to be installed consistently when setting up the project on another development system.

Visual Studio Code can be used as the primary development environment for writing and managing the project source code. It provides features such as syntax highlighting, code navigation, integrated terminal support, error identification, and debugging assistance. These features make it suitable for working with React, JavaScript, CSS, and related project files.

Axios is used as the HTTP communication library in the frontend application. It provides an API integration layer through which the React application can communicate with backend services. The project contains an API service configuration that uses an API URL and can attach authentication tokens to requests. This provides a foundation for connecting the frontend with the planned Node.js/Express backend.

html5-qrcode is used to support the QR-code scanning functionality of the portal. It allows the application to access a compatible device camera and scan QR codes during event check-in. This feature is intended to make attendee attendance verification faster and reduce dependence on completely manual verification procedures.

Chart.js and **react-chartjs-2** are used for the analytics dashboard. Chart.js provides charting capabilities, while react-chartjs-2 allows Chart.js functionality to be used within React components. These tools help present event-related information in a graphical form so that organizers can understand attendance and other available event statistics more easily.

HTML, CSS, and JavaScript form the fundamental technologies of the web interface. HTML provides the basic structure of web content, CSS is used for styling and layout, and JavaScript provides the application logic and interactive behavior. The project also uses CSS styling for elements such as cards, forms, tables, buttons, badges, and dashboard components.

For backend and database development, the project presentation specifies **Node.js/Express and MySQL** as the intended technologies. Express can be used to develop the server-side API, while MySQL can provide persistent storage for users, events, registrations, attendance records, and other application data. However, the submitted project source primarily contains the frontend implementation, so these backend and database technologies represent the planned integration rather than fully implemented components in the current source.

Overall, these development tools and frameworks provide a modular foundation for the **Event Management and Analytics Portal**. React handles the user interface, Node.js/npm supports the development environment, Axios provides API communication, html5-qrcode supports QR-based check-in, and Chart.js supports analytics visualization. The planned Express and MySQL integration can further extend the application by providing server-side processing and persistent data storage.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The **Event Management and Analytics Portal** uses a web-based front-end design developed with **React.js**. The front end provides the main interaction between users and the application. It is designed to provide separate interfaces for different types of users, including organizers, staff, and attendees. The interface focuses on simple navigation, clear presentation of event information, and easy access to the major functions of the portal.

The application contains several important pages and components. The **Login** and **Registration** pages provide the entry points for users to access the portal. After authentication, users are directed to the appropriate parts of the application according to their assigned role. The **Event List** page allows users to view available events and provides event-related information. The interface also supports searching and filtering of events so that users can locate relevant events more easily.

For organizers, the **Create Event** page provides an interface for entering event details such as the event name, category, date, location, and capacity. The organizer can use this interface to create and manage event information. Role-based route protection is used so that functions intended only for organizers are not directly accessible to other users. This provides a structured interface where different users can access features according to their responsibilities.

The front end also includes an **event registration** functionality for attendees. Users can select an event and register for participation. The **QR Check-In** page provides a separate interface for attendance verification. Staff or authorized organizers can scan an attendee's QR code using the device camera through the `html5-qrcode` library. A manual ticket-code option is also provided as an alternative method of verifying an attendee when QR scanning is not used.

The **Analytics Dashboard** provides organizers with a graphical representation of available event information. The project uses **Chart.js** and **react-chartjs-2** to display analytics in the form of charts. This allows event-related information to be presented visually instead of only through plain text or tables. The **Audit Logs** page provides organizers with information about recorded activities such as event creation and check-in actions, helping provide visibility into important system activities.

The interface also contains a **navigation bar** and a **notification banner** that help users interact with the application. The navigation structure provides access to the features available to the current user, while notifications can be used to communicate important information to users. The application uses CSS styling for components such as cards, forms, buttons, tables, badges, input fields, and dashboard elements. A grid-based layout is used in several areas to organize information clearly.

The front-end architecture also includes reusable components such as **AuthContext**, **ProtectedRoute**, **Navbar**, and **NotificationBanner**. The authentication context manages the user's authentication-related information, while protected routes control access to pages based on authentication and user roles. This component-based structure makes the application easier to maintain because common interface and access-control logic can be reused across multiple pages.

The front end is also prepared for communication with a backend API through **Axios**. The API service contains configuration for an API URL and supports sending authentication information with requests. In the current submitted project, the main application functionality is implemented on the frontend using client-side state and sample data. The planned Node.js/Express backend and MySQL database can later be connected through this API layer to provide persistent storage and server-side processing.

Overall, the front-end design of the **Event Management and Analytics Portal** provides a structured and user-friendly interface for event discovery, event creation, registration, attendance verification, audit monitoring, and analytics. The React-based component structure also provides a foundation for extending the application with additional features and connecting it to the planned backend and database in future development.

3.2 Back-End Design

The **Event Management and Analytics Portal** is designed with a backend architecture that is intended to provide server-side processing, API communication, authentication, and persistent data management. According to the project design, **Node.js with Express.js** is planned as the backend technology. The backend acts as an intermediate layer between the React-based frontend and the MySQL database. It is responsible for receiving requests from the frontend, processing the required operations, and returning appropriate responses.

The backend is intended to expose **RESTful APIs** for the major operations of the application. Through these APIs, the frontend can request event information, create events, register attendees, process check-in information, retrieve audit records, and obtain data required for analytics. The frontend uses **Axios** as the API communication layer, with the API URL configured through the application environment. This structure separates the user interface from the server-side logic and makes it possible to change or extend the backend without redesigning the complete frontend.

Authentication and authorization are important parts of the planned backend design. The system supports different roles such as **Organizer, Staff, and Attendee**, and access to application functions is controlled according to these roles. The frontend already contains authentication-related components such as `AuthContext` and `ProtectedRoute`, while the planned backend can provide server-side authentication and authorization. Authentication tokens can be transferred through the API using the **Bearer token** mechanism, allowing the server to identify authenticated users and control access to protected operations.

The backend is also intended to manage event-related operations. When an organizer creates an event, the corresponding information can be received through an API request and stored in the database. Similarly, event details can be retrieved when users open the event listing. Registration-related requests can be processed by the backend so that attendee information and event participation records can be stored and retrieved when required. This approach allows the application to move from temporary client-side data to persistent data storage.

The planned backend also supports the **QR-based check-in process**. When staff or an authorized organizer scans an attendee's QR code, the check-in information can be sent to the backend through an API. The server can validate the registration and record the attendance information in the database. A manual ticket-code verification mechanism can also be handled through the same API structure. This provides a common server-side mechanism for maintaining attendance information.

Another planned responsibility of the backend is maintaining **audit logs**. Important operations such as event creation and attendee check-in can be recorded along with relevant information about the activity. These records can later be requested by authorized organizers through the audit-log interface. The backend therefore provides a suitable location for maintaining activity records and supporting accountability within the portal.

The backend is also designed to support the **Analytics Dashboard**. Instead of keeping analytics information only as static or sample data in the frontend, the planned server can retrieve event, registration, and attendance records from the database and provide summarized information through

APIs. The React frontend can then use this information with Chart.js to display charts and other visual representations.

The planned database layer uses **MySQL** for persistent storage. The database can store information related to users, roles, events, registrations, attendance, and audit activities. Node.js/Express can communicate with the MySQL database through suitable database connectivity mechanisms. This separation between the frontend, backend, and database provides a structured architecture that can be expanded as the number of users and events increases.

It is important to note that the **submitted project source primarily contains the React frontend implementation**. A complete Node.js/Express server implementation, MySQL schema, and database connection were not present in the submitted source. Therefore, the backend described in this section represents the **planned backend design and intended integration** rather than a fully implemented backend in the current version. The existing Axios API layer provides a foundation for connecting the frontend to this backend in a future development stage.

Overall, the proposed backend design provides a clear separation between the presentation layer and server-side processing. The combination of **React frontend → Axios API layer → Node.js/Express backend → MySQL database** provides the intended architecture for authentication, event management, registration, QR check-in, audit logging, and analytics. This design also provides a foundation for making the **Event Management and Analytics Portal** more persistent, secure, maintainable, and scalable in future versions.

3.3 Database Design

The **Event Management and Analytics Portal** is designed to use a relational database for storing and managing application data. The project presentation specifies **MySQL** as the intended database technology. The database is planned to provide persistent storage for information related to users, events, registrations, attendance, and audit activities. This allows information generated through the portal to be maintained systematically and retrieved whenever required.

The database design is intended to organize the application's information into separate logical entities. A **User** entity can store details required for identifying users and their roles in the system. The role information is important because the portal supports different types of users, including **Organizer, Staff, and Attendee**. By maintaining user and role information in the database, the system can associate users with the operations they are authorized to perform.

An **Event** entity is intended to store the details of events created through the portal. These details can include the event name, category, date, location, and capacity, which are the types of information represented by the current frontend event model. Event records can be associated with the organizer who created them. This structure allows event information to be retrieved and displayed on the event-listing interface.

A **Registration** entity can be used to maintain information about attendees registering for events. It can establish a relationship between a user and an event and can be used to identify which users are registered for particular events. Registration information is also important for the attendance verification process because the QR code or ticket code used during check-in can be associated with the corresponding registration.

An **Attendance or Check-In** entity can store information about attendee attendance at an event. When an authorized staff member or organizer verifies an attendee through QR scanning or manual ticket-code

entry, the check-in information can be recorded. This information can later be used by the analytics functionality to provide event attendance-related information.

The database design also includes an **Audit Log** concept for maintaining records of important activities performed within the system. Activities such as event creation and attendee check-in can be recorded so that organizers can review system activity. The audit information can contain details about the activity, the user who performed it, and the associated event or operation. This supports monitoring and accountability within the portal.

The planned relationships between these entities provide a structured flow of information. An organizer can create multiple events, an event can have multiple registrations, and registered attendees can have corresponding attendance records. Users can also be associated with audit activities based on the actions they perform. These relationships reduce unnecessary duplication of information and make it easier to retrieve related records.

The database is also intended to support the **Analytics Dashboard**. Event, registration, and attendance information stored in the database can be queried and summarized to produce useful statistics. These results can then be sent through the backend API to the React frontend, where they can be displayed using Chart.js and react-chartjs-2. This provides a connection between stored application data and the analytical interface.

However, the **submitted project source does not contain a completed MySQL database schema, SQL scripts, or database connection implementation**. The current implementation primarily uses React frontend state and sample data. Therefore, the database structure described here represents the **planned database design** based on the project architecture and presentation, rather than claiming that these MySQL tables are already implemented in the submitted source.

Overall, the planned MySQL database provides the persistent data layer for the **Event Management and Analytics Portal**. The intended structure separates users, events, registrations, attendance, and audit information into related entities. When integrated with the planned Node.js/Express backend, the database will allow the portal to move from temporary frontend data to persistent storage and support reliable event management, attendance tracking, audit monitoring, and analytics.

3.4 Version Control

Version control is an important part of the development process of the **Event Management and Analytics Portal** because it helps developers maintain different versions of the project source code and track changes made during development. The project can use **Git** as the version control system for maintaining the frontend source code, configuration files, dependencies, and other project-related files.

Git allows developers to save changes as separate commits during the development process. Each commit represents a particular stage of development and can contain changes such as adding a new page, modifying an existing component, fixing an error, or improving the user interface. This makes it easier to identify when a particular change was introduced and to return to an earlier working version if required.

For team-based development, Git can also support collaboration between multiple team members. Different members can work on different parts of the application and combine their changes into the main project through branches and merging. For example, one member can work on event management functionality while another works on the analytics dashboard or authentication-related components. This helps organize the development process and reduces the risk of directly overwriting another member's work.

The project source code can be maintained in a **Git repository**, which provides a central location for storing and managing the project files. A platform such as **GitHub** can be used to host the repository and provide additional features such as repository sharing, version history, branch management, and collaboration. The repository can also be used to maintain the project documentation and supporting files when required.

Version control is particularly useful for the **Event Management and Analytics Portal** because the application contains multiple frontend components and features, including authentication, event management, registration, QR check-in, audit logs, and analytics. As these features are developed and modified, Git provides a systematic way to track the changes and maintain different versions of the application.

In the future, version control can also support the planned backend and database integration. Separate branches can be used for backend development, database integration, bug fixes, and new features before the changes are merged into the main version. This provides a structured development workflow and makes it easier for the team to maintain the project as it grows.

Overall, Git-based version control provides a reliable way to **track, manage, and collaborate on changes** in the Event Management and Analytics Portal. It helps maintain project history, supports team collaboration, and provides a safer development process when new features or modifications are introduced.

3.5 APIs or External Services

The **Event Management and Analytics Portal** is designed to use APIs and external libraries to support communication between different parts of the application and to provide additional functionality. The main API-related component present in the submitted project is the **Axios-based API service**, which provides a structured way for the React frontend to communicate with a backend server. The project also uses external libraries for QR-code scanning and data visualization.

The **Axios API service** is used as the communication layer between the frontend and the planned backend. The application contains an API service file that is configured to use an API base URL through the `REACT_APP_API_URL` environment variable, with a local API address available as the default. This allows the frontend to send requests to backend endpoints without directly depending on a fixed server address. The API layer can be used for operations such as retrieving event information, creating events, processing registrations, handling check-in information, and retrieving audit or analytics data.

The project also includes support for **authentication tokens** in API requests. The Axios configuration can retrieve a stored token and attach it to requests using the Bearer authorization format. This provides the foundation for secure communication between the frontend and a future backend authentication system. The backend can use the token to identify the requesting user and apply the appropriate role-based permissions.

For QR-based attendance verification, the project uses the **html5-qrcode** library. This library provides QR-code scanning functionality through a compatible device camera. It is used in the QR Check-In section of the application to allow authorized staff or organizers to scan attendee QR codes. A manual ticket-code verification option is also available as an alternative to QR scanning.

For analytics visualization, the project uses **Chart.js** together with **react-chartjs-2**. These libraries allow information available to the analytics dashboard to be represented through charts and graphical components. The use of these libraries makes it easier for organizers to understand event-related information visually rather than depending only on text or tabular data.

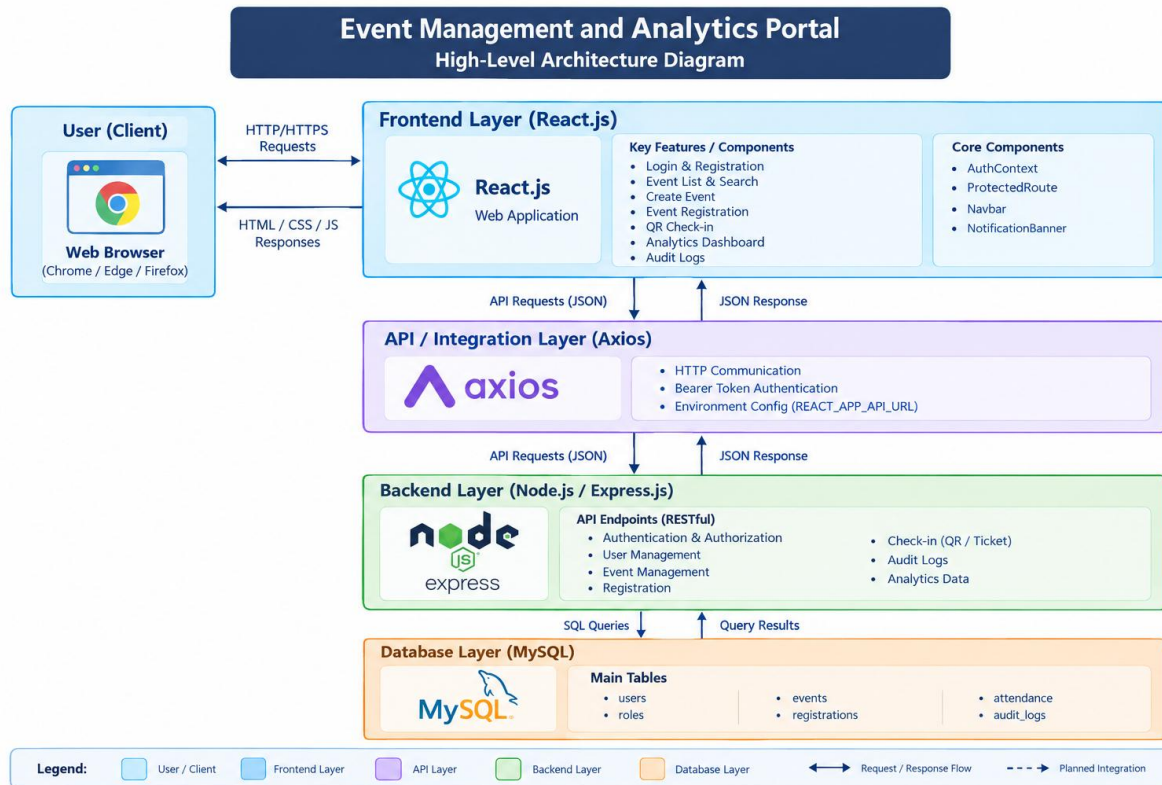
The project presentation specifies **Node.js/Express** as the intended backend technology. The Express backend is planned to expose APIs that handle server-side operations and communicate with the MySQL database. The React frontend can access these services through Axios. This creates a clear communication flow between the user interface, backend services, and database.

It is important to distinguish between the APIs and libraries already present in the submitted project and the planned external services. The submitted source contains the **frontend API integration layer and external frontend libraries**, but a complete Express backend and MySQL database service were not found in the submitted source. Therefore, the backend APIs and database services should be considered planned integration rather than currently implemented external services.

Overall, the API and external-service design provides an extensible foundation for the **Event Management and Analytics Portal**. Axios provides frontend-to-server communication, html5-qrcode supports QR-based check-in, and Chart.js supports analytics visualization. The planned Node.js/Express APIs and MySQL database can later provide persistent server-side functionality and allow the portal to operate as a complete full-stack application.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



4.2 Description of Each Layer (Frontend, Backend, Database)

The Frontend Layer of the Event Management and Analytics Portal is developed using React.js and is responsible for providing the main user interface. It includes pages such as Login, Registration, Event List, Create Event, QR Check-In, Analytics Dashboard, and Audit Logs. Users interact with these pages through a web browser, while components such as Navbar, AuthContext, ProtectedRoute, and NotificationBanner help manage navigation, authentication, access control, and notifications.

The frontend also provides important application features such as event search, event registration, QR-based attendance verification, and analytics visualization. The html5-qrcode library is used for QR scanning, while Chart.js and react-chartjs-2 are used to display graphical information in the analytics dashboard. These components provide the interface through which organizers, staff, and attendees can perform their respective activities.

The API/Integration Layer uses Axios to provide communication between the React frontend and the planned backend. Axios is configured to communicate with an API URL and can attach authentication information using a Bearer token. Through this layer, the frontend can exchange information with backend services for operations such as event management, registration, check-in, audit logs, and analytics.

The Backend Layer is planned using Node.js and Express.js. It is intended to handle server-side processing, authentication and authorization, event management, registration, attendance verification, audit logging, and

analytics-related operations. The submitted project mainly contains the React frontend and Axios integration, while the complete Node.js/Express backend is planned for future integration.

The Database Layer is planned using MySQL to provide persistent storage for users, roles, events, registrations, attendance, and audit logs. The planned backend will communicate with the database using SQL queries and provide the required information to the frontend through APIs. The submitted source does not contain the completed MySQL database implementation, so this layer represents the planned database architecture for the future full-stack version of the Event Management and Analytics Portal.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The **Event Management and Analytics Portal** can be deployed in a local development environment using a computer with Node.js, npm, and a modern web browser. The React application can be started locally using the development server and accessed through a web browser. This setup is suitable for development, testing, debugging, and demonstrating the current frontend implementation.

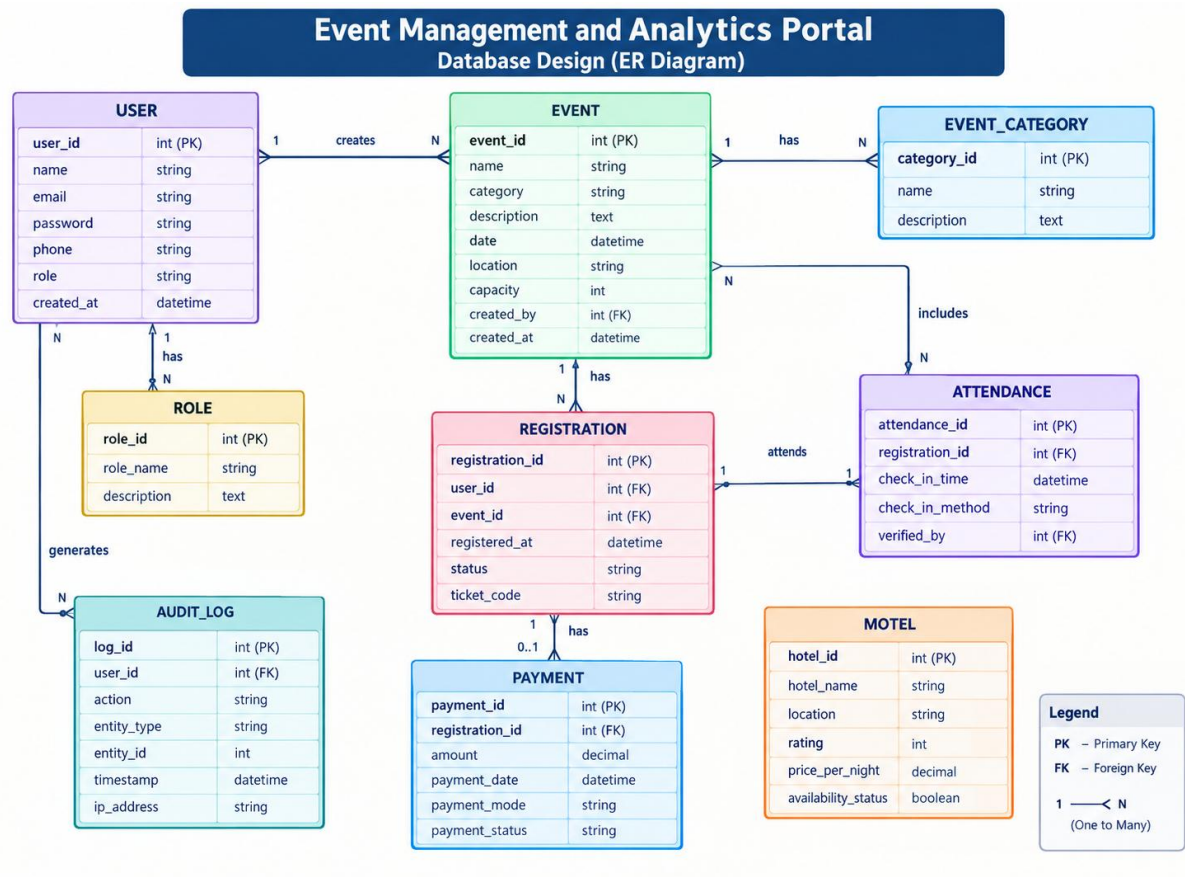
For a complete deployment, the **React frontend** can be hosted on a cloud or web hosting platform, allowing users to access the application through a browser. The planned **Node.js/Express backend** can be deployed on a server or cloud environment and connected to the frontend through API requests. This architecture allows organizers, staff, and attendees to access the portal remotely without installing the application on their individual systems.

The planned **MySQL database** can also be deployed on a server or managed cloud database service. It would store persistent information such as users, events, registrations, attendance, and audit logs. The backend would communicate with the database, while the frontend would communicate with the backend through the API layer. In the current submitted project, the backend and MySQL deployment are planned components rather than implemented deployment services.

A **CI/CD pipeline** can be introduced to automate the process of building, testing, and deploying the application. A Git-based repository can be connected to a CI/CD service so that changes pushed by developers can automatically trigger dependency installation, application building, testing, and deployment steps. This would make future updates more organized and reduce the need for repeated manual deployment activities.

Overall, the deployment architecture provides three possible environments: **local deployment for development and testing, cloud deployment for user access, and CI/CD automation for future continuous development and deployment**. The current project primarily represents the frontend application, while cloud hosting, the complete backend, MySQL deployment, and CI/CD automation can be incorporated when the planned full-stack version of the portal is implemented.

CHAPTER -5 DESIGN



CHAPTER -6 IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The **Event Management and Analytics Portal** is implemented as a React-based web application consisting of multiple modules, with each module responsible for a specific event-management function. The main modules include **Login and Registration, Event Management, Search and Filter, Event Registration, QR Check-In, Notifications, Audit Logs, and Analytics Dashboard**. These modules are connected through React routing, shared state, and reusable components.

The **Login and Registration Module** allows users to enter their basic information and select their role as Organizer, Staff, or Attendee. The current implementation creates a client-side session and stores user and token information using browser local storage. The **ProtectedRoute** component is used to restrict access to pages based on the logged-in user's role.

The **Event Management Module** allows organizers to create events by entering details such as event title, category, date, venue, and capacity. After an event is created, it is added to the application's event state and an audit record is generated. The **Event List and Search Module** displays available events and provides title-based searching and category filtering, allowing users to find relevant events more easily.

The **Event Registration and QR Check-In Modules** support attendee participation and attendance verification. Attendees can register for events through the event interface, with registration information maintained in client-side state. For check-in, authorized Organizer or Staff users can scan a QR code using the `html5-qrcode` library or enter a ticket code manually. Successful check-in activity is displayed and added to the audit records.

The **Audit Logs and Analytics Modules** provide monitoring and reporting functionality. The Audit Logs module displays recorded activities such as event creation and QR check-in in a table containing time, user role, and action details. The Analytics Dashboard displays summary information along with bar and pie charts using **Chart.js** and **react-chartjs-2**. The current analytics values are based on the application's sample/client-side data, while persistent database-backed analytics are planned for future backend integration.

Section 6.2 – Front-End Logic (UI Rendering and State Management)

The **Event Management and Analytics Portal** uses **React.js** to manage the front-end logic and dynamically render the user interface. React components are responsible for displaying different pages and updating the interface based on user actions. The application uses components such as Login, Register, EventList, CreateEvent, QRCheckIn, AuditLogs, and AnalyticsDashboard to separate the functionality into manageable sections.

The application uses **React state management** to maintain information such as events, registrations, audit logs, and user-related data during application execution. When a user performs an operation, such as creating an event or completing a check-in, the corresponding state is updated and React automatically re-renders the affected interface. This allows users to see updated information without manually refreshing the page.

Authentication-related state is handled through the **AuthContext** component. It provides authentication information to different parts of the application and helps determine whether a user is logged in. The

ProtectedRoute component uses this information to control access to pages based on authentication and user roles. For example, organizer-specific pages such as event creation, analytics, and audit logs are protected from unauthorized access.

The front-end also uses event handlers to respond to user interactions such as button clicks, form submissions, searching, filtering, registration, and QR check-in. Form inputs are connected to component state so that entered values can be captured and processed. The **Navbar** and **NotificationBanner** components provide common interface functionality across the application, while CSS styling controls the layout and visual presentation of cards, forms, tables, buttons, and other UI elements.

The application also contains an **Axios-based API service** that provides a connection point between the React interface and the planned backend. Currently, the submitted implementation primarily uses client-side state and sample data, while the complete backend and database integration are planned for future development. This front-end structure allows the application to be extended later with persistent server-side data while keeping the user interface and state-management logic organized.

6.3 API Integration

The **Event Management and Analytics Portal** includes an API integration layer that allows the React frontend to communicate with backend services. The project uses **Axios** for handling HTTP requests. The Axios service is configured with an API base URL through the `REACT_APP_API_URL` environment variable, with a local API address available as the default configuration.

The API integration is designed to support operations such as retrieving event information, creating events, handling registrations, processing check-in information, and obtaining audit and analytics data. This separates the user interface from the server-side processing and allows the frontend components to communicate with backend services in a structured manner.

The application also includes authentication support in the Axios configuration. A stored authentication token can be retrieved and attached to outgoing requests using the **Bearer token** authorization format. This provides a mechanism for the backend to identify authenticated users and apply appropriate access permissions based on their roles.

In the submitted project, the **Axios API service and frontend integration structure are present**, but a complete Node.js/Express backend was not included in the source. Therefore, the API integration currently provides the foundation for backend communication, while the complete server-side API and MySQL database integration are planned for future development.

Overall, the API integration provides a clear communication path between the **React frontend and the planned backend services**. It also makes the application easier to extend because additional API endpoints can be connected without significantly changing the existing user-interface structure.

6.4 Authentication and Authorisation

The **Event Management and Analytics Portal** includes an authentication and authorisation mechanism to control access to different parts of the application. The system provides **Login** and **Registration** pages through which users can enter their account information and access the portal. After authentication, the application maintains user-related information so that the appropriate interface and features can be provided.

The application supports different user roles, including **Organizer, Staff, and Attendee**. Role-based access is implemented on the frontend using the AuthContext and ProtectedRoute components. AuthContext manages authentication-related information, while ProtectedRoute checks whether a user is authenticated and whether the user's role is permitted to access a particular page.

Different roles have access to different functions. **Organizers** can access features such as event creation, analytics, and audit logs. **Staff and Organizers** can access the QR check-in functionality, while attendees can access event-related functions intended for participants. This role-based structure helps prevent users from directly accessing features that are not intended for their role.

The application also stores authentication-related information on the client side and can maintain a token for API communication. The Axios API service is designed to attach the stored token to requests using the **Bearer token** format. This provides a foundation for implementing server-side authentication and authorization when the planned Node.js/Express backend is integrated.

In the current submitted project, authentication and role-based access are primarily handled on the **frontend**, and a complete server-side authentication system was not included in the source. Therefore, backend-level authentication, secure password management, and database-backed user verification remain part of the planned full-stack implementation.

CHAPTER 7- FEATURES

7.1 List of Main Features

The **Event Management and Analytics Portal** provides a user registration and login facility that allows users to create an account and securely access the application. The login process maintains the user's session using a token stored in the browser. The portal supports different user roles, including **Organizer, Staff, and Attendee**, allowing the application to provide different functionalities based on the role of the logged-in user.

The portal provides **role-based access control** to manage access to different sections of the application. Organizers can access features such as event creation, analytics, and audit logs, while Staff members can access the check-in functionality. Attendees can view available events and register for them. This separation helps organize the responsibilities of different users within the event management system.

The **Event Creation and Event Listing** features allow organizers to create and manage event information through the web interface. An organizer can enter details such as the event title, category, date, venue, and maximum capacity. Once created, the event is displayed in the event listing, where users can view the available event details.

The application also provides **Event Search and Filtering** functionality. Users can search for events based on their title and filter events according to their category. The current application includes categories such as Technology, Workshops, and Management. This feature makes it easier for users to find relevant events when multiple events are available.

The **Event Registration** feature allows attendees to register for events through the portal. The application maintains the registration status and updates the user interface based on the selected event. This provides a simple mechanism for attendees to indicate their participation in an event.

The portal includes **QR-Based Check-In** functionality for recording attendee attendance. The application uses the `html5-qrcode` library to scan QR codes through the browser. When a QR code is successfully scanned, the ticket information is displayed and the check-in action is recorded. A **manual ticket-code check-in** option is also provided as an alternative to QR scanning.

A **Notification Banner** is provided to display important messages to users within the application. The notification area can communicate relevant information or status messages while users are working with different sections of the portal. This provides a simple communication mechanism within the application.

The **Audit Logs** feature records important activities performed in the portal. Actions such as event creation and QR-based check-in can generate audit records containing information such as the timestamp, user role, and action performed. This allows organizers to view and track activities carried out within the system.

The **Analytics Dashboard** provides a visual representation of event-related information. It displays values such as total registrations, average check-in rate, and active managed events. The dashboard also provides charts showing registered and checked-in participants across events and the distribution of events according to their categories.

Finally, the application includes an **API Integration Layer** using Axios. This layer prepares the frontend for communication with backend REST APIs and provides a structured way for the React application to exchange data with backend services. In the current submitted implementation, the backend and database integration are prepared as an extension of the application rather than being fully implemented.

7.2 How Each Feature Works

1. User Registration and Login

The User Registration and Login feature provides the basic authentication mechanism for the Event Management and Analytics Portal. New users can create an account by entering the required information through the registration page. After registration, users can log in using their credentials. When the login is successful, the application stores the authentication token in the browser's local storage. The `AuthContext` manages the authentication information and makes it available to different components of the application. This allows the portal to identify the logged-in user and maintain the user's session while navigating between different pages.

2. Role-Based Access Control

The Role-Based Access Control feature manages access to different functionalities according to the user's role. The application supports roles such as Organizer, Staff, and Attendee. The `ProtectedRoute` component checks the user's authentication status and role before allowing access to protected pages. For example, organizers can access event creation, analytics, and audit-log pages, while organizers and staff can access the check-in page. This provides separate functionality for different types of users and prevents unauthorized access to role-specific sections.

3. Event Creation

The Event Creation feature allows organizers to add new events to the portal. The organizer enters information such as event title, category, date, venue, and maximum capacity through a dedicated form. After the form is submitted, the application creates an event object and adds it to the current event data. The newly created event can then be viewed in the event listing. The application also generates an audit entry for the event creation activity, allowing the action to be tracked.

4. Event Listing

The Event Listing feature displays the available events in an organized format. Each event contains important information such as its title, category, date, venue, and capacity. Users can view the available events from the event-list page and select events according to their requirements. This feature acts as the main interface for users to discover the events available through the portal.

5. Event Search and Filtering

The Event Search and Filtering feature helps users find specific events more easily. Users can enter an event title or related text into the search field to locate matching events. The application also provides category-based filtering, allowing users to select categories such as Technology, Workshops, Management, or All. The event list is updated according to the selected search or filter criteria, making it easier to locate relevant events.

6. Event Registration

The Event Registration feature allows attendees to register for available events. After selecting an event, the attendee can use the registration option provided by the application. The registration status is maintained in the application's client-side state, and the interface is updated according to the user's registration. This provides a simple mechanism for attendees to indicate their participation in an event. In the current implementation, registration information is not permanently stored in a database.

7. QR-Based Check-In

The QR-Based Check-In feature is used to record attendee attendance during an event. The application uses the `html5-qrcode` library to provide QR-code scanning functionality through the browser. Staff members or organizers can scan the participant's QR or ticket code. After a successful scan, the application displays the ticket information and records the check-in action. An audit entry is also generated for the check-in activity, providing a record of the attendance operation.

8. Manual Ticket-Code Check-In

The Manual Ticket-Code Check-In feature provides an alternative method when QR scanning is not used. A staff member or organizer can manually enter the participant's ticket code into the provided input field. The application processes the entered code as part of the check-in workflow. This feature is also useful for testing the check-in functionality without requiring a physical QR code.

9. Notification Banner

The Notification Banner feature provides a common area for displaying important messages within the application. The banner is available as part of the main application interface and can communicate relevant information or status messages to users. This provides a simple way of presenting notifications without requiring users to navigate to a separate page. The feature can also be extended in the future for event reminders and registration-related notifications.

10. Audit Logs

The Audit Logs feature records important activities performed within the portal. Actions such as event creation and attendee check-in can generate audit records. The audit information includes details such as the timestamp, user role, and action performed. Authorized users can view these records through the Audit Logs page in a structured table. This provides basic activity tracking and allows organizers to review important operations performed in the application.

11. Analytics Dashboard

The Analytics Dashboard provides a visual summary of event-related information. It displays values such as total registrations, average check-in rate, and the number of active managed events. The dashboard also contains charts that represent registered and checked-in participants across events and the distribution of events according to their categories. Chart.js and react-chartjs-2 are used to display the graphical information. The current implementation uses client-side or sample data for the analytics.

12. API Integration

The API Integration feature provides the communication structure required between the React frontend and backend services. The application uses Axios to prepare HTTP requests and responses between the frontend and the planned REST API. The API configuration can use the REACT_APP_API_URL environment variable, with a default API address available in the application. An authentication token can also be attached to requests as a Bearer token. The current source contains the frontend API integration structure, while the complete Node.js/Express backend and MySQL database integration are planned extensions.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman is commonly used for testing and verifying communication between a frontend application and backend REST APIs. It allows developers to send HTTP requests such as **GET, POST, PUT, and DELETE** to API endpoints and examine the returned responses. For the Event Management and Analytics Portal, Postman can be used to validate the API layer that is prepared in the React frontend through Axios.

In the current submitted project, the **React frontend contains an Axios service** in `src/services/api.js`. This service is configured with a default API base URL of `http://localhost:5000/api` and can also use the `REACT_APP_API_URL` environment variable. The Axios service is prepared to attach the stored authentication token as a **Bearer Authorization header** when an API request is made. This provides the communication structure required for connecting the frontend with a backend API.

The frontend itself is tested primarily through the browser because the submitted source does not contain a complete Node.js/Express backend. Therefore, Postman testing of actual backend endpoints cannot be claimed as completed for the current submitted implementation. The documentation should distinguish between the **prepared API integration** in the frontend and the **future backend API testing** that can be performed once the Express server is implemented.

For the planned backend, Postman can be used to test user-related API operations such as registration and login. A request can be sent to the appropriate endpoint with the required user information in JSON format. The response can then be checked to verify whether the server returns the expected status code and response data. Authentication testing can also verify whether a valid token is generated and whether the token can be supplied in subsequent requests.

Postman can also be used to test the planned **Event Management APIs**. For example, an event creation request can send details such as event title, category, date, venue, and capacity to the backend. The response can be checked to confirm that the event was successfully processed. Similar API requests can later be used to retrieve event information, update event details, or delete events when these backend operations are implemented.

The **Registration and Check-In APIs** can also be verified through Postman after backend integration. Registration requests can contain attendee and event information, while check-in requests can contain ticket or participant information. The returned response can be checked to ensure that the request was processed correctly. This testing can help verify that the frontend and backend exchange information in the expected format.

Postman can additionally be used to test the **Audit Log and Analytics APIs** planned for the backend. Audit requests can be checked to ensure that important activities such as event creation and attendee check-in are recorded correctly. Analytics requests can be tested to verify that the backend returns the required registration, attendance, and event-category information for display in the React analytics dashboard.

For frontend-backend integration testing, the React application would send requests through Axios to the backend API, while Postman would independently send requests to the same API endpoints. The responses obtained through Postman can be compared with the data expected by the frontend. This helps identify problems such as incorrect endpoint URLs, invalid request formats, authentication issues, missing fields, or unexpected response structures.

The project documentation identifies browser testing, npm test, npm run build, manual form testing, and QR-capable camera testing as the current testing approaches. It also states that **no backend integration tests are present in the submitted source**. Therefore, Postman should be documented as a tool for the planned backend/API testing rather than presenting unverified backend test results as completed results.

Overall, Postman provides a useful testing environment for the future Node.js/Express backend of the Event Management and Analytics Portal. It can verify API requests, response data, authentication headers, status codes, and communication between the frontend and backend. Once the backend and MySQL persistence layer are implemented, Postman can be used to systematically test the complete API workflow before connecting the services to the React frontend.

8.2 Integration Testing

Integration testing is performed to verify that the different modules of the **Event Management and Analytics Portal** work correctly when they are connected with each other. Instead of testing each feature independently, integration testing checks whether information and actions are correctly passed between related components. In the current frontend implementation, integration-level checks focus on the connection between authentication, routing, event management, check-in, audit logs, and analytics.

The first integration check is between the **Authentication and Protected Routes**. When a user logs in, the authentication information is made available through AuthContext. The protected routes then use this information to determine whether the user is allowed to access a particular page. For example, an Organizer should be able to access the Create Event, Analytics, and Audit Logs pages, while a Staff user should have access to the Check-In page. This verifies that authentication information is correctly connected with the application's navigation and access-control logic.

The **Event Creation and Event Listing** modules are also integrated with each other. When an Organizer enters the required event information and submits the Create Event form, the new event is added to the application's event state. The Event List then uses this updated information to display the newly created event. This integration verifies that the data entered through the creation form is correctly reflected in the event listing.

Integration testing is also performed between **Event Search/Filtering and Event Data**. The search and category filter operate on the available event information and update the displayed results according to the user's input. Testing this connection verifies that entering a search term or selecting a category produces the expected changes in the event list. This ensures that the filtering logic works correctly with the event data maintained by the application.

The **Event Registration and Event State** connection can also be checked during integration testing. When an attendee registers for an event, the registration status should be reflected in the application's interface. The integration check verifies that the registration action correctly updates the relevant client-side state and that the event interface displays the appropriate registration status. In the current implementation, this information remains client-side rather than being persisted in a backend database.

Another important integration check is between the **QR Check-In and Audit Logs** modules. When a QR code or manual ticket code is successfully processed, the check-in action should result in an appropriate audit entry. The Audit Logs page can then display the recorded activity. This verifies that the check-in module and audit-log functionality correctly communicate through the application's state management.

The **Event Creation and Audit Logs** modules are similarly connected. When an Organizer successfully creates an event, the application generates an audit record describing the event creation action. The

integration test checks whether this record is correctly added and subsequently displayed on the Audit Logs page. This provides a connection between event management and activity tracking.

Integration testing also covers the **Analytics Dashboard and Event Information**. The dashboard uses the available application data to display summary values and charts related to registrations, check-ins, and event categories. Testing verifies that the dashboard components render correctly and receive the required information. The current analytics implementation uses client-side or sample data, so database-level analytics integration is not part of the submitted implementation.

The **Axios API Integration Layer** is another integration point prepared in the application. The Axios service provides a communication path between the React frontend and a planned backend API. It can use the configured API base URL and attach the stored authentication token as a Bearer Authorization header. However, because the submitted project does not contain a complete Node.js/Express backend, full frontend-to-backend integration testing cannot be claimed as completed.

Overall, integration testing confirms that the major frontend modules work together as expected. The current project focuses on verifying route protection, authentication context, event creation and listing, filtering, registration, QR/manual check-in, audit-log updates, and analytics rendering. The project documentation specifies that integration-level checks should verify that the main routes connect to their corresponding pages, authentication context is available to required components, and state changes in **CreateEvent** and **QRCheckIn** are reflected in **EventList** and **AuditLogs**.

8.3 Tools Used for Testing

The **Event Management and Analytics Portal** uses different testing tools and methods to verify the functionality, usability, and reliability of the application. Since the current submitted project is primarily a React frontend, the testing process focuses mainly on browser-based functional testing, application behavior, build verification, and manual testing. The following tools are used for testing the application.

1. Browser Developer Tools

Browser Developer Tools are used to inspect the application's user interface, identify console errors, and observe client-side behavior. They help in checking whether React components are rendering correctly and whether any errors occur while navigating between pages. Developer Tools can also be used to inspect browser storage, including the authentication token and other client-side information maintained by the application.

2. npm Test

The **npm test** command is used to run the testing functionality provided by the React project setup. It can be used to execute the configured React test scripts and identify problems in the application's components or logic. The project package includes the standard Create React App testing script, although no extensive backend integration tests are included in the submitted source.

3. npm Build

The **npm run build** command is used to verify whether the React application can be successfully compiled for production. A successful build indicates that the application source code and required dependencies can be processed without compilation errors. This is also useful before deployment because it verifies that the frontend can generate a production-ready build.

4. Chrome / Microsoft Edge

Web browsers such as **Google Chrome and Microsoft Edge** are used for functional testing of the portal. The application pages, forms, navigation, protected routes, event search, filtering, event creation, registration, analytics, and audit logs can be manually checked through the browser. Browser testing also helps verify that the user interface behaves correctly during normal interaction.

5. QR-Capable Camera

A **QR-capable camera** is used to test the QR check-in functionality. The Check-In page uses the html5-qrcode library to scan QR codes through the browser. Testing with a camera verifies whether a QR code can be detected successfully and whether the corresponding ticket information and audit entry are displayed after a successful check-in.

6. Manual Form Testing

Manual form testing is used to verify input fields and user interactions throughout the application. Forms such as registration, login, event creation, search, filtering, and manual ticket-code check-in can be tested by entering different values and observing the resulting application behavior. This helps identify incorrect validation, unexpected results, or interface problems.

7. Postman

Postman can be used for testing REST API communication when the planned Node.js/Express backend is implemented. It allows developers to send API requests and inspect response status codes and data. In the current submitted source, the Axios API layer is prepared for backend communication, but a complete backend is not included. Therefore, full backend API testing using Postman is considered part of the planned integration testing rather than a completed test of the submitted source.

8. Axios API Layer

The **Axios** service in the frontend provides the prepared communication layer for API requests. It uses a configurable API base URL and can attach the stored authentication token as a Bearer Authorization header. Although Axios is primarily an integration component rather than a standalone testing tool, it is important for verifying how the frontend will communicate with the planned backend services.

Overall, these tools and testing methods provide coverage for the major functionality currently available in the project. The project documentation specifically identifies **Browser Developer Tools, npm test, npm run build, Chrome/Edge, QR-capable camera, and manual form testing** as the primary testing tools and approaches for the submitted frontend implementation.

8.4 Test Cases and Results

The **Event Management and Analytics Portal** was tested using a set of functional test cases to verify the correct behaviour of its major modules and features. The testing started by opening the /events route to confirm that the Events page correctly displayed the available sample events. The authentication and role-based access functionality was then tested by logging in with different user roles. When an Organizer logged in, the application correctly provided access to the **Create Event, Analytics, and Audit Logs** pages, while a Staff user was able to access the **QR Check-In** functionality without accessing Organizer-only pages. The event search functionality was tested by entering the search term “**Tech**”, and the matching event remained visible in the event list. Category filtering was also tested by selecting the **Technology** category, after which the corresponding technology event was displayed. The Event Creation feature was tested by creating a new event as an Organizer, and the expected result was that the newly created event appeared in the event list and an audit entry was generated for the action. The QR and manual check-in functionality was tested by providing a ticket code, after which the ticket result was displayed and a corresponding audit entry was added. The **Analytics Dashboard** was then opened to verify that the summary cards and charts were rendered correctly. The **Audit Logs** page was tested using an Organizer account, and the recorded activities were displayed in the audit table. Finally, protected-page access was tested using a user without the required role, and the application redirected the user to the Events page instead of allowing access to the restricted functionality. Based on these functional checks, the tested scenarios were marked as **Pass**. These results describe the behaviour supported by the submitted source structure and should be re-run in the target development environment before final institutional submission, particularly if backend or database integration is added later.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The deployment of the Event Management and Analytics Portal involves preparing the application environment, installing the required dependencies, configuring the application, and running the frontend in a web browser. Since the submitted project is primarily a React-based frontend application, the current deployment process focuses on deploying the client-side application. The project can be deployed locally for development and demonstration, while the planned Node.js/Express backend and MySQL database can be connected when the complete backend implementation is available.

Step 1: Prepare the Environment:

First, install the required software such as Node.js, npm, and a modern web browser such as Google Chrome or Microsoft Edge. The project source code is extracted into a suitable folder on the system. The developer should open the project folder using Visual Studio Code or another suitable development environment and verify that the frontend source files and package.json are available.

Step 2: Install Project Dependencies:

Open the terminal inside the project directory and run the required npm installation command. This installs the React libraries and other packages used by the application, including Axios, Chart.js, react-chartjs-2, and html5-qrcode. After installation is completed, the project is ready to be started locally.

Step 3: Configure the Application:

The API integration layer uses the environment variable REACT_APP_API_URL when it is provided. Otherwise, the application uses http://localhost:5000/api as the default API base URL. Therefore, if a backend server is available, the appropriate API URL should be configured before deployment. In the submitted version, the backend is not included, so the frontend can be demonstrated independently using its available client-side functionality.

Step 4: Start the Application:

The React application can be started using the npm start command. The development server compiles the application and makes it available through the local browser. The developer can then open the displayed local address and verify that the Login, Registration, Event List, Event Creation, QR Check-In, Analytics, and Audit Log pages are loading correctly.

Step 5: Test the Deployed Application:

After starting the application, the main features should be tested in the browser. User registration and login, role-based navigation, event creation, event search and filtering, event registration, QR/manual check-in, audit logs, and analytics should be checked. Browser Developer Tools can also be used to identify JavaScript errors, routing problems, or API-related issues.

Step 6: Production Deployment:

For production deployment, the React application can be built into optimized static files using the appropriate npm build command. These generated files can then be hosted on a suitable web hosting or cloud platform. If the planned Node.js/Express backend and MySQL database are implemented, the frontend API URL must be changed to point to the deployed backend, and the backend must be configured to communicate with the MySQL database.

Overall, the deployment process consists of preparing the environment, installing dependencies, configuring the API connection, starting or building the React application, testing the major features, and finally hosting the generated application. The submitted project currently supports the frontend

deployment process, while complete backend and database deployment would be performed after their implementation.

9.2 Environment Configuration

The environment configuration of the Event Management and Analytics Portal involves preparing the system with the required software, project dependencies, application settings, and runtime environment. Proper configuration ensures that the application can be executed without dependency or compatibility issues. The project is primarily developed as a React-based web application and can be configured for local development and demonstration using a standard computer system.

The first step is to install **Node.js and npm**, which are required to manage and run the React application. After installing Node.js, the project source code is opened in Visual Studio Code or another suitable development environment. The terminal is then opened inside the project directory so that the required project commands can be executed.

The project dependencies are maintained through the package.json file. Running the npm installation command downloads and installs the libraries required by the application. The project uses several supporting packages, including **Axios** for API communication, **Chart.js** and **react-chartjs-2** for displaying analytics charts, and **html5-qrcode** for QR-code scanning. Installing these dependencies ensures that the corresponding application modules can function correctly.

The application also requires a suitable **web browser** for execution and testing. Google Chrome or Microsoft Edge can be used to access the application after the React development server is started. Browser Developer Tools are useful during configuration because they allow developers to inspect console errors, network requests, application behavior, and other runtime information.

The API environment is configured through the file src/services/api.js. The application checks for the REACT_APP_API_URL environment variable to determine the backend API address. If this variable is not provided, the application uses http://localhost:5000/api as the default API URL. This approach makes it possible to change the backend address without modifying the main application components.

For local development, the application can be started using the React development command after all dependencies have been installed. The application is then opened through the local development-server address in a web browser. During this stage, the login, registration, event listing, event creation, search and filtering, QR check-in, audit logs, and analytics interfaces can be checked to ensure that the configured environment is working properly.

The project also requires appropriate browser permissions for features that depend on device capabilities. In particular, the QR check-in module uses the device camera to scan QR codes. Therefore, when the application is tested using a laptop or other camera-enabled device, the browser may request permission to access the camera. Granting the required permission allows the QR scanning functionality to operate.

The environment configuration also supports the separation between the frontend and the planned backend. The submitted source contains the React frontend and an Axios-based API integration layer, while a complete Node.js/Express backend and MySQL database are not included in the submitted project package. Therefore, the backend server and database configuration cannot be described as currently implemented. When these components are integrated, their connection details can be provided through appropriate environment variables and API configuration.

For production deployment, the environment can be configured using the production API address instead of the local localhost address. The React application can then be built into optimized production files and hosted on a suitable web server or cloud platform. The backend, when implemented, can be hosted separately and connected to the MySQL database using its corresponding configuration settings.

Overall, the environment configuration provides the basic setup required to run, test, and deploy the Event Management and Analytics Portal. It includes the Node.js and npm environment, project dependencies, browser configuration, API URL configuration, camera permission for QR scanning, and provisions for future backend and database integration.

9.3 Hosting of Frontend and Backend

The Event Management and Analytics Portal is designed as a web-based application in which the frontend provides the user interface and the backend is intended to provide server-side processing and data management. The frontend is developed using React and contains the major user-facing modules such as login, registration, event management, event search, QR check-in, audit logs, and analytics. The backend is planned using Node.js and Express, with MySQL intended for persistent database storage.

The **frontend hosting** involves deploying the React application to a web server or cloud hosting platform that can serve the generated static files. Before hosting, the React application is built into optimized production files. The frontend can then be accessed by users through a standard web browser. During deployment, the API configuration must point to the appropriate backend address instead of the local development address.

The frontend communicates with the backend through the Axios integration layer available in the project. The API service uses the configured `REACT_APP_API_URL` value and falls back to `http://localhost:5000/api` when no environment variable is provided. In a hosted environment, this configuration can be changed to the URL of the deployed backend server. This allows the same frontend structure to communicate with different backend environments.

The **backend hosting** is planned for the Node.js and Express server. The backend would receive requests from the React frontend, process authentication and event-related operations, and provide responses through REST APIs. It would also communicate with the MySQL database for storing and retrieving persistent information such as users, events, registrations, attendance records, and audit information.

The backend and database can be hosted separately from the frontend. In such an architecture, the React frontend is responsible for presentation and user interaction, while the Express backend acts as the application server and MySQL provides persistent data storage. Communication between the frontend and backend would take place through HTTP/HTTPS API requests, with the backend returning the required data to the frontend.

For a production environment, the frontend should use the deployed backend API address, while the backend should be configured with the appropriate database connection details. Environment variables can be used to store configuration values such as API addresses and database connection parameters rather than placing them directly inside application source code. This makes the deployment configuration easier to manage across development and production environments.

In the **submitted project**, the React frontend and Axios API integration are available, but a complete Node.js/Express backend and MySQL implementation are not included in the source package. Therefore, actual production hosting of the backend and database cannot be claimed as completed for the current version. The hosting architecture described above represents the planned deployment structure for the complete system.

Overall, the intended hosting architecture separates the frontend, backend, and database into their respective layers. The React frontend provides browser-based access, the Node.js/Express backend handles application services and API requests, and MySQL provides persistent storage. This structure provides a clear path for moving the current frontend implementation toward a complete hosted Event Management and Analytics Portal.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

During the development of the Event Management and Analytics Portal, several challenges were encountered while designing and implementing the different modules. Since the project contains multiple interconnected features such as authentication, role-based access, event management, QR check-in, audit logging, analytics, and API integration, maintaining proper coordination between these components required careful implementation and testing.

One of the main challenges was implementing **user authentication and role-based authorisation**. Different users such as Organizer, Staff, and Attendee require different levels of access. The application therefore needed to ensure that organizer-specific pages such as event creation, analytics, and audit logs were not directly accessible to other roles. Implementing the AuthContext and ProtectedRoute components helped manage authentication information and restrict access according to the user's role.

Another issue was managing **application state between different modules**. Event information is used by multiple parts of the application, including the event listing, search and filtering functionality, registration process, and analytics. Similarly, check-in activities need to generate corresponding audit-log entries. Maintaining these state updates correctly was important so that changes made in one module were reflected in other relevant parts of the application.

The **QR check-in functionality** also introduced additional development challenges. The application uses the html5-qrcode library to access the device camera and scan QR codes. Camera permissions, scanner behavior, and successful scan handling need to be managed properly. To support testing when a physical QR code or camera is not convenient, the application also provides a manual ticket-code entry option.

Another challenge was implementing **search and filtering** in the event listing. Users should be able to search events based on their titles and filter them using categories. The application needs to update the displayed event list according to the selected search term or category without affecting the original event information. Proper state handling and filtering logic were therefore required to provide consistent results.

The **analytics dashboard** required additional attention because it contains summary information and graphical representations. Chart.js and react-chartjs-2 are used to display the charts. The data needs to be prepared in a format that can be consumed by the chart components, while the summary values and visualizations must remain consistent with the available event information.

Another issue involved the **API integration layer**. The project includes Axios configuration for communicating with a backend API and supports an environment-based API URL. However, the submitted project package does not contain a complete Node.js/Express backend or MySQL database implementation. As a result, the frontend API integration had to be designed so that it could support backend communication when the server-side components are integrated in the future.

Finally, **testing and debugging** were important throughout development. Different user roles, protected routes, event creation, search and filtering, QR/manual check-in, audit logs, and analytics needed to be checked individually and together. Browser Developer Tools were useful for identifying runtime issues, while manual testing helped verify the behavior of the application's main features. These development challenges helped improve the overall structure of the application and provided a clearer foundation for future backend, database, and production deployment integration.

10.2 Solutions Applied

To address the issues encountered during the development of the Event Management and Analytics Portal, several solutions were implemented across the different modules. The solutions mainly focused on

improving role management, maintaining consistent application state, handling QR check-in, implementing search and filtering, displaying analytics, and preparing the application for future backend integration.

The **authentication and role-based access issue** was addressed using the AuthContext and ProtectedRoute components. Authentication information is maintained through the application context, while protected routes check the user's role before allowing access to specific pages. Organizer-only features such as event creation, analytics, and audit logs are restricted to the Organizer role, while the QR check-in functionality is available to the appropriate Staff and Organizer roles.

To solve the problem of **maintaining event and audit information**, React state management was used within the application. When an organizer creates an event, the event is added to the available event information and a corresponding audit entry is generated. Similarly, when a successful QR check-in is performed, the check-in information is displayed and an audit record is added. This provides a consistent connection between event activities and the audit-log module.

The **QR check-in problem** was handled using the html5-qrcode library. The QR scanner accesses the device camera and processes the scanned ticket information. The application also provides a manual ticket-code entry option, which acts as an alternative method for testing or when scanning is not convenient. This makes the check-in module easier to test under different conditions.

The **search and filtering issue** was addressed by implementing search and category-based filtering in the event listing module. Users can enter a search term to find events based on their titles and can select a category to narrow down the displayed events. The filtering logic works with the available event state so that the displayed results are updated according to the user's selection.

For the **analytics functionality**, Chart.js and react-chartjs-2 were used to provide graphical representations of available event information. Summary values and charts are displayed through the Analytics Dashboard, making the information easier to understand. The analytics module is currently based on the available application data and provides a foundation for connecting the dashboard to persistent database information in the future.

The **API integration challenge** was addressed by creating a separate Axios service in src/services/api.js. This provides a centralized location for API communication and uses the REACT_APP_API_URL environment variable when configured. The service also supports adding the stored authentication token as a Bearer token in requests. This structure allows the frontend to be connected to a Node.js/Express backend when the backend implementation is integrated.

Finally, **testing and debugging** were handled by testing the major application routes and features individually and together. Browser Developer Tools were used to identify runtime issues, while manual testing was performed for login, role restrictions, event creation, event search and filtering, QR/manual check-in, audit logs, and analytics. These solutions helped create a more organized frontend implementation while also keeping the application ready for future backend and MySQL database integration.

10.3 Current Limitations or Known Bugs

The current version of the Event Management and Analytics Portal has some limitations because the submitted project is primarily a React-based frontend implementation. The main application features are available for demonstration and testing, but some components that would be required for a complete production system are not yet implemented.

The most important limitation is that a **complete backend server is not included** in the submitted project. The frontend contains an Axios-based API integration layer and is prepared to communicate with a Node.js/Express backend, but the complete server-side implementation is not available in the submitted source. Therefore, backend operations cannot currently provide full server-side processing and persistent data management.

Another limitation is the **absence of the MySQL database implementation** in the submitted source. Event, registration, attendance, and audit information are currently handled through the application's available client-side state and sample data. As a result, data may not provide permanent storage across application sessions in the same way that a database-backed system would.

The **analytics dashboard** is also limited by the available data. The dashboard provides summary information and charts, but the submitted implementation does not contain a complete database-backed analytics system. Therefore, the displayed analytics are based on the application's available data rather than continuously generated statistics from a production database.

The current **authentication and authorisation mechanism** is implemented on the frontend using the authentication context, protected routes, stored token information, and role-based navigation. Since a complete backend authentication system is not included, server-side validation of users and permissions is not available in the submitted version. A production implementation would require backend-side authentication and authorisation checks.

The **QR check-in module** depends on browser camera access and the availability of a compatible device. Camera permissions and browser support can affect QR scanning. The application therefore includes a manual ticket-code option for testing and alternative check-in input. This provides a workaround but does not replace a complete backend-based ticket validation system.

The application also currently uses **sample event and audit information** for demonstration. Because persistent backend storage is not included, changes made during a session may not be permanently maintained after the application is restarted. This is an important limitation for a real-world event management system where event registrations, attendance records, and audit information would need to remain available.

Overall, the current limitations are mainly related to the incomplete backend and database integration rather than the basic frontend functionality. The submitted version provides the main user interface and client-side features, while persistent storage, server-side authentication, database-backed analytics, and complete API processing remain areas for further development.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The Event Management and Analytics Portal can be further enhanced with several features to make it more suitable for real-world event management. The planned improvements mainly focus on completing the backend and database integration, improving security, providing persistent data storage, and expanding the analytics capabilities of the existing application.

One major planned feature is the **complete Node.js and Express backend integration**. The backend can provide REST APIs for user authentication, event creation, event registration, check-in, audit logs, and analytics. This will allow the frontend to communicate with a centralized server instead of depending mainly on client-side state.

Another planned feature is **MySQL database integration** for persistent data storage. User details, event information, registrations, attendance records, and audit logs can be stored in the database. This will ensure that information remains available even after the application is restarted and will provide a proper foundation for managing a larger number of events and users.

The application can also be enhanced with **server-side authentication and authorization**. Authentication can be handled securely through the backend, while role-based permissions can be validated at the server level. This would provide an additional security layer for Organizer, Staff, and Attendee accounts and prevent unauthorized API access.

A further planned feature is **database-backed analytics and reporting**. Instead of relying on available client-side or sample data, the analytics dashboard can retrieve real-time information from the database. Organizers could use this information to monitor registrations, attendance, event participation, and other relevant event statistics.

The **event registration and QR check-in system** can also be expanded. Future implementation can include database-based ticket generation and validation, allowing each attendee's registration and attendance status to be securely maintained. This would make the check-in process more suitable for actual events with larger numbers of participants.

Another planned improvement is **enhanced notifications and communication**. The portal can be extended to provide event reminders, registration confirmations, check-in notifications, and other event-related messages. This can help organizers communicate important information to registered attendees more efficiently.

The system can also be improved with **advanced event management features**, such as updating and cancelling events, managing event capacity, and maintaining more detailed event information. These features can provide organizers with greater control over events throughout their lifecycle.

Finally, the application can be prepared for **production deployment and scalability**. The React frontend, Node.js/Express backend, and MySQL database can be deployed as separate components and configured to communicate through secure APIs. These planned improvements would transform the current frontend-focused implementation into a more complete, persistent, and production-oriented Event Management and Analytics Portal.

11.2 Possible Integrations or Optimisations

The Event Management and Analytics Portal can be further improved by integrating additional services and optimizing the existing application. These improvements can make the system more efficient, secure, scalable, and suitable for handling a larger number of users and events.

One possible integration is a **complete Node.js and Express backend** with the existing React frontend. The Axios API layer already provides a structured way for the frontend to communicate with backend services. Integrating the backend would allow operations such as user authentication, event management, registration, check-in, audit logging, and analytics to be processed through REST APIs.

Another important integration is a **MySQL database** for persistent storage. The database can store users, roles, events, registrations, attendance records, and audit logs. This would remove the dependency on temporary client-side data and allow information to remain available across different sessions and devices.

The system can also be integrated with **email or notification services**. Such services could be used to send registration confirmations, event reminders, ticket information, and other important notifications to attendees. This would reduce the need for organizers to communicate event information manually.

The **QR check-in system** can be optimized by connecting QR codes with database records. When a QR code is scanned, the backend could verify the associated ticket, check the registration status, prevent duplicate check-ins, and store the attendance time. This would provide a more reliable check-in process for larger events.

The **analytics module** can be optimized by obtaining data directly from the database and generating reports dynamically. Additional statistics such as total registrations, attendance percentage, event capacity utilization, and participation trends could be displayed through charts and reports. This would make the analytics dashboard more useful for organizers.

Performance optimization can also be applied to the frontend. Components can be organized to reduce unnecessary rendering, while API requests can be handled efficiently. For larger event lists, pagination or optimized search and filtering can be introduced so that the application does not need to process a large amount of data at once.

Security can be improved by implementing **server-side authentication and authorization**, secure password handling, token expiration, input validation, and protected API endpoints. These improvements would complement the existing frontend role-based access mechanism and provide stronger protection when the system is connected to a real backend.

Finally, the complete system can be optimized for **cloud deployment and scalability**. The React frontend, Express backend, and MySQL database can be deployed as separate services and configured through environment variables. With proper API design, database indexing, validation, and efficient data handling, the portal can be extended to support more events, users, and attendance records while maintaining reliable performance.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The **Event Management and Analytics Portal** is a web-based application designed to simplify the process of creating, managing, registering for, and monitoring events through a centralized interface. The project provides different functionalities for users based on their roles, including Organizer, Staff, and Attendee. The application brings together important event-related activities such as user registration and login, event creation, event listing, event search and filtering, event registration, QR-based check-in, audit logging, and analytics within a single portal.

The application is developed primarily using **React** for the frontend. The frontend provides an interactive interface through which users can navigate between the different modules of the system. Authentication information is managed using the application context, while protected routes are used to control access to role-specific functionality. Organizers can access features such as event creation, analytics, and audit logs, while Staff users can perform check-in activities. This role-based structure helps organize the functionality according to the responsibilities of different users.

The project also provides an **event management module** through which organizers can create events by entering information such as event title, category, date, venue, and capacity. Created events can be displayed through the event listing interface, where users can search and filter events. The project also includes event registration functionality and a QR-based check-in module using `html5-qrcode`. A manual ticket-code option is additionally provided for testing and situations where QR scanning is not convenient.

Another important part of the project is the **audit and analytics functionality**. Activities such as event creation and successful check-in can generate audit records that can be viewed through the Audit Logs page. The Analytics Dashboard presents summary information and graphical representations using `Chart.js` and `react-chartjs-2`. These features provide organizers with a way to observe event-related information through a structured interface.

The project also includes an **Axios-based API integration layer** that provides a communication structure for connecting the frontend with backend services. The application supports an environment-based API URL and uses a Bearer token for API requests. However, the submitted project package primarily contains the React frontend, and a complete Node.js/Express backend and MySQL database implementation are not included. Therefore, persistent database storage and complete server-side processing are planned extensions rather than fully implemented components of the current version.

Overall, the project provides a functional frontend foundation for an Event Management and Analytics Portal. It combines event management, role-based access, registration, QR check-in, audit logging, and analytics into a single web application. The planned integration of a Node.js/Express backend and MySQL database can further extend the system by providing persistent storage, server-side authentication, database-backed analytics, and a stronger foundation for production deployment.

12.2 What Was Achieved

The development of the Event Management and Analytics Portal successfully resulted in a functional React-based web application that brings several important event management activities together in a single platform. The project established the basic structure required for users to interact with events through different modules, providing a foundation for further backend and database integration.

A **user registration and login system** was implemented along with role-based access for Organizer, Staff, and Attendee users. The application uses authentication context and protected routes to control access to different sections of the portal. Organizer-specific functionality such as event creation, analytics, and audit logs is separated from the functionality available to other users.

The **event management functionality** was successfully developed. Organizers can create events by entering details such as the event title, category, date, venue, and capacity. The application also provides an event listing interface with search and category-based filtering, allowing users to locate relevant events more easily.

The project also achieved the implementation of **event registration and QR-based check-in functionality**. The QR check-in module uses the html5-qrcode library to support camera-based scanning. A manual ticket-code option is also available for testing. Successful check-in activities can be reflected in the application's audit information.

The **audit logging and analytics modules** were also developed. The Audit Logs page provides information about recorded activities such as event creation and check-in operations. The Analytics Dashboard uses Chart.js and react-chartjs-2 to present summary information and graphical representations, providing organizers with a clearer view of the available event data.

The project further achieved the creation of an **Axios-based API integration structure**. The frontend is configured to communicate with a backend through a configurable API URL and can attach authentication information to requests using a Bearer token. This provides a foundation for connecting the application to a complete server-side implementation in the future.

Finally, the project established a clear foundation for future development. The submitted implementation successfully demonstrates the major frontend workflows and user interface features, while the complete Node.js/Express backend and MySQL database remain planned extensions. The achieved work therefore provides the core frontend structure and functionality required to continue developing the portal into a complete production-oriented event management system.

12.3 Skills Learned During Development

During the development of the Event Management and Analytics Portal, several technical and practical skills were developed through the design, implementation, testing, and documentation of the application. Working on different modules helped in understanding how individual frontend components can be combined to form a complete web-based application.

One of the major skills learned was **React frontend development**. The project provided practical experience in creating React components, designing user interfaces, handling forms, managing application state, and connecting different pages through routing. The development of pages such as Login,

Registration, Event List, Create Event, QR Check-In, Audit Logs, and Analytics Dashboard helped in understanding how different components work together within a React application.

The project also provided experience in **authentication and role-based access control**. Implementing AuthContext and protected routes helped in understanding how authentication information can be maintained and how access to specific application features can be controlled according to user roles. This provided practical knowledge of separating functionality for Organizer, Staff, and Attendee users.

Another important skill developed was **state management and event handling**. The project required maintaining event information, handling form submissions, updating event lists, processing search and filtering operations, and recording audit activities. This helped in understanding how changes in one part of a React application can affect other components and how application state can be managed to support interactive functionality.

The development of the **QR check-in module** provided practical experience with integrating an external library. The html5-qrcode library was used to access the device camera and process QR codes. This helped in understanding how browser-based applications can interact with device capabilities and how external packages can be incorporated into a project.

The project also helped develop skills in **data visualization** through the use of Chart.js and react-chartjs-2. Creating the Analytics Dashboard provided experience in preparing data for charts and presenting information in a visual format. This demonstrated how graphical representations can make event-related information easier to understand.

Another skill learned was **API integration using Axios**. The project includes a centralized API service that can communicate with a backend using HTTP requests and can attach authentication information through a Bearer token. This provided an understanding of how a React frontend can be structured for communication with backend services and how environment variables can be used for API configuration.

The development process also improved **testing and debugging skills**. Browser Developer Tools, manual feature testing, and application build and test commands were used to identify and verify application behavior. Testing different user roles, protected routes, event creation, search and filtering, QR check-in, audit logs, and analytics helped develop a systematic approach to finding and resolving issues.

Finally, the project improved **teamwork, project organization, problem-solving, and technical documentation skills**. Working with multiple modules required planning the application structure, identifying development issues, testing individual features, and documenting the implementation and limitations. Overall, the project provided practical experience in frontend development, authentication, state management, API integration, external library usage, data visualization, testing, debugging, and software project documentation.

CHAPTER -13 REFERENCES

Books

Books

Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

Duckett, J. (2015). *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley.

Freeman, E., & Robson, E. (2020). *Head First JavaScript Programming*. O'Reilly Media.

Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.

Tutorials and Learning Resources

W3Schools. (2025). *HTML, CSS, JavaScript, and React Tutorials*. Retrieved from <https://www.w3schools.com>

MDN Web Docs. (2025). *Web Technologies Documentation*. Retrieved from <https://developer.mozilla.org>

freeCodeCamp. (2025). *Responsive Web Design and JavaScript Development Resources*. Retrieved from <https://www.freecodecamp.org>

GeeksforGeeks. (2025). *React, JavaScript, Web Development, and Database Tutorials*. Retrieved from <https://www.geeksforgeeks.org>

TutorialsPoint. (2025). *React, JavaScript, Node.js, Express, and Web Development Tutorials*. Retrieved from <https://www.tutorialspoint.com>

APIs and Tools Used

React. (2025). *React Documentation – A JavaScript Library for Building User Interfaces*. Retrieved from

[React Documentation](#)

Node.js. (2025). *Node.js Documentation – JavaScript Runtime Environment*. Retrieved from

[Node.js Documentation](#)

Express.js. (2025). *Express.js Web Framework Documentation*. Retrieved from

[Express.js Documentation](#)

Axios. (2025). *Axios – Promise Based HTTP Client*. Retrieved from

[Axios Documentation](#)

Chart.js. (2025). *Chart.js – JavaScript Charting Library*. Retrieved from

[Chart.js Documentation](#)

Html5-qrcode. (2025). *HTML5 QR Code Scanner Documentation*. Retrieved from

[Html5-qrcode Documentation](#)

MySQL. (2025). *MySQL 8.0 Reference Manual*. Retrieved from

[MySQL Documentation](#)

npm. (2025). *npm Documentation – Package Manager for JavaScript*. Retrieved from

[npm Documentation](#)

Documentation and Research References

Microsoft. (2025). *Visual Studio Code Documentation*. Retrieved from

[Visual Studio Code Documentation](#)

Git. (2025). *Git Documentation – Distributed Version Control System*. Retrieved from

[Git Documentation](#)

Chart.js. (2025). *Data Visualization and Charting Documentation*. Retrieved from

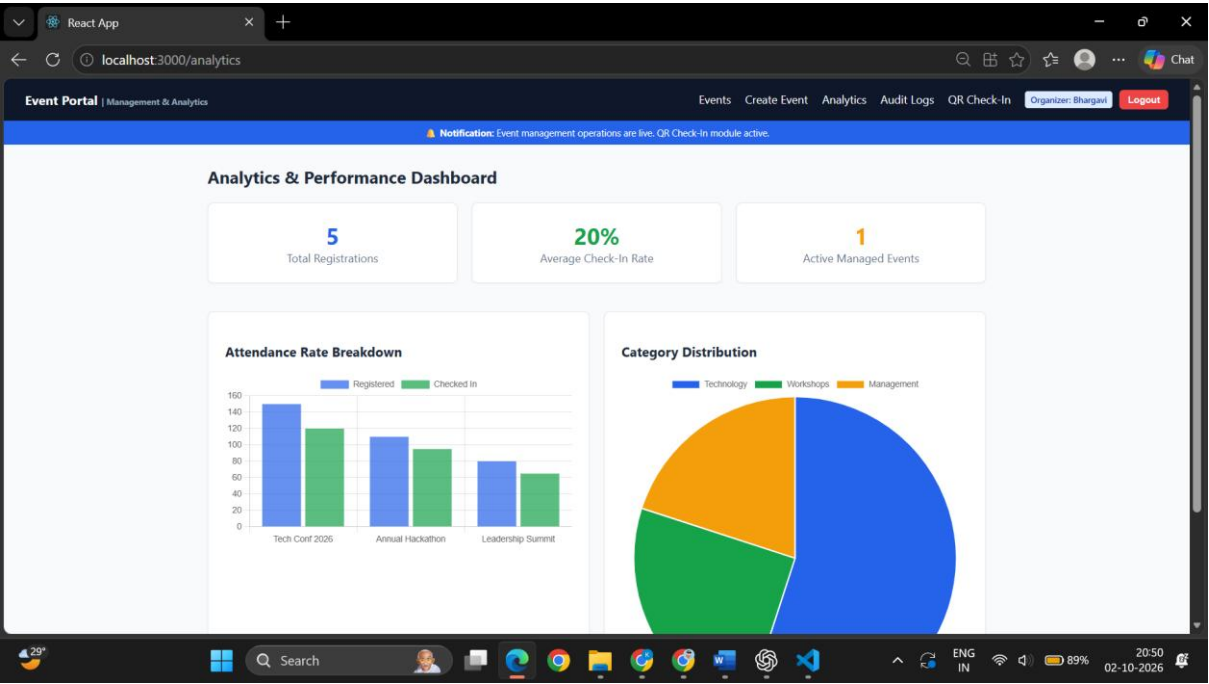
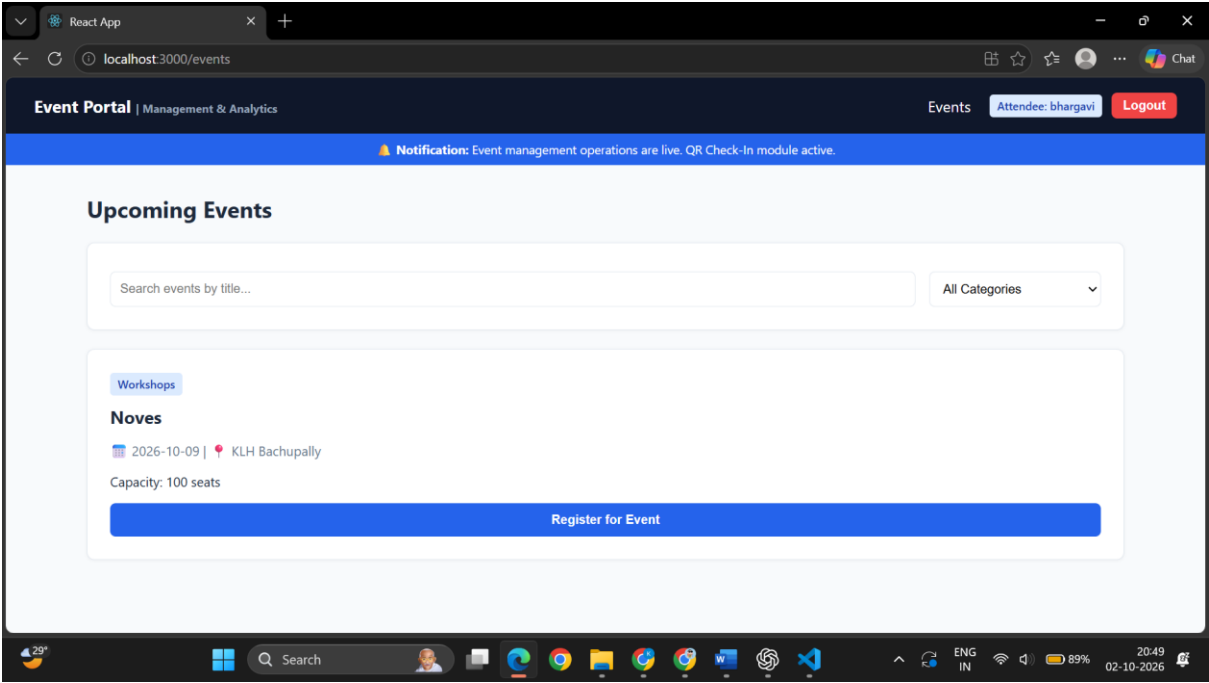
[Chart.js Documentation](#)

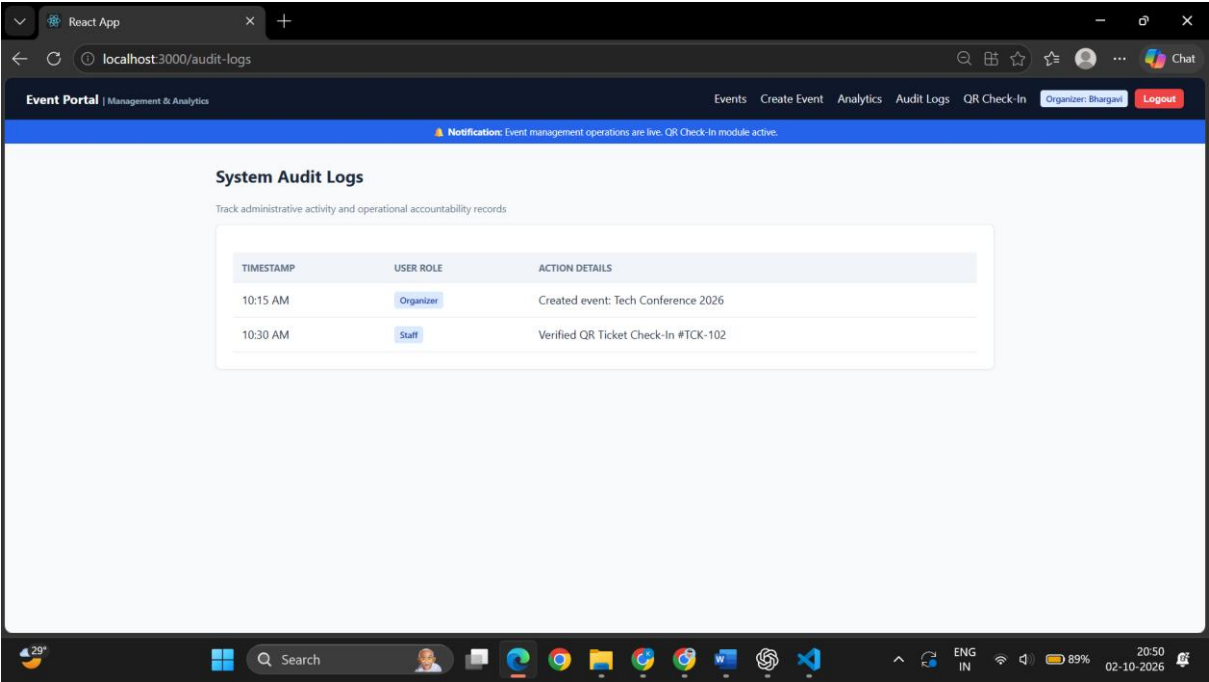
React. (2025). *React Router and Application Development Documentation*. Retrieved from

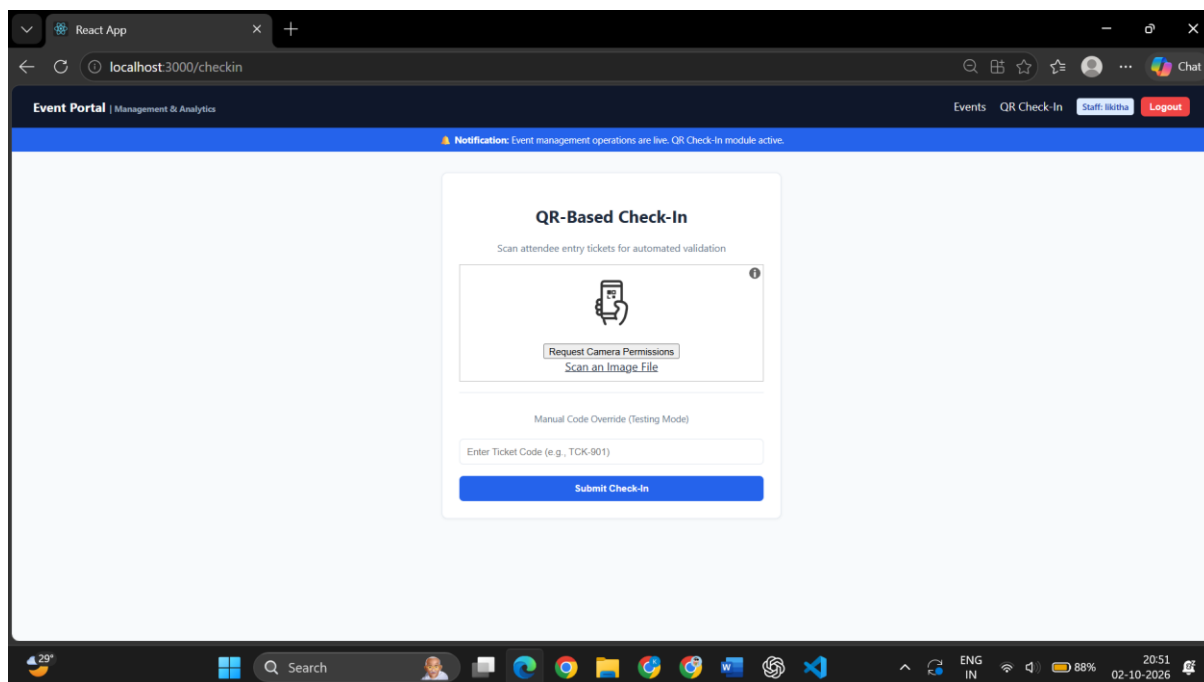
[React Documentation](#)

CHAPTER -14 APPENDICES

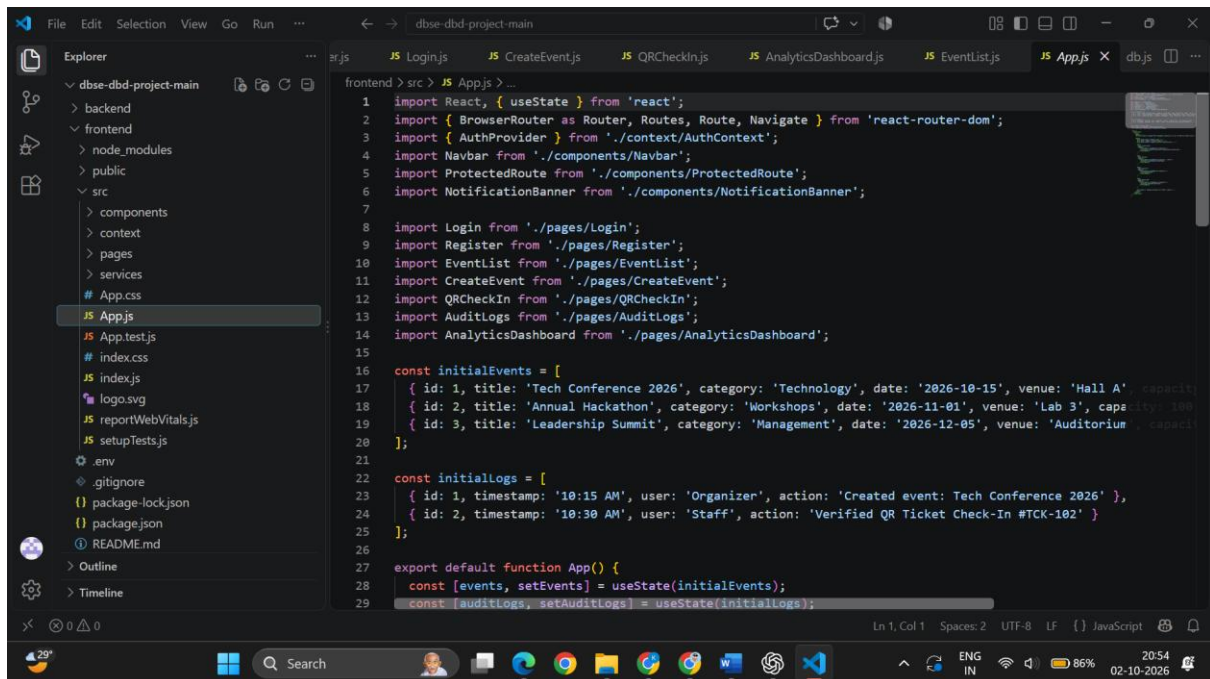
Screenshots of the app





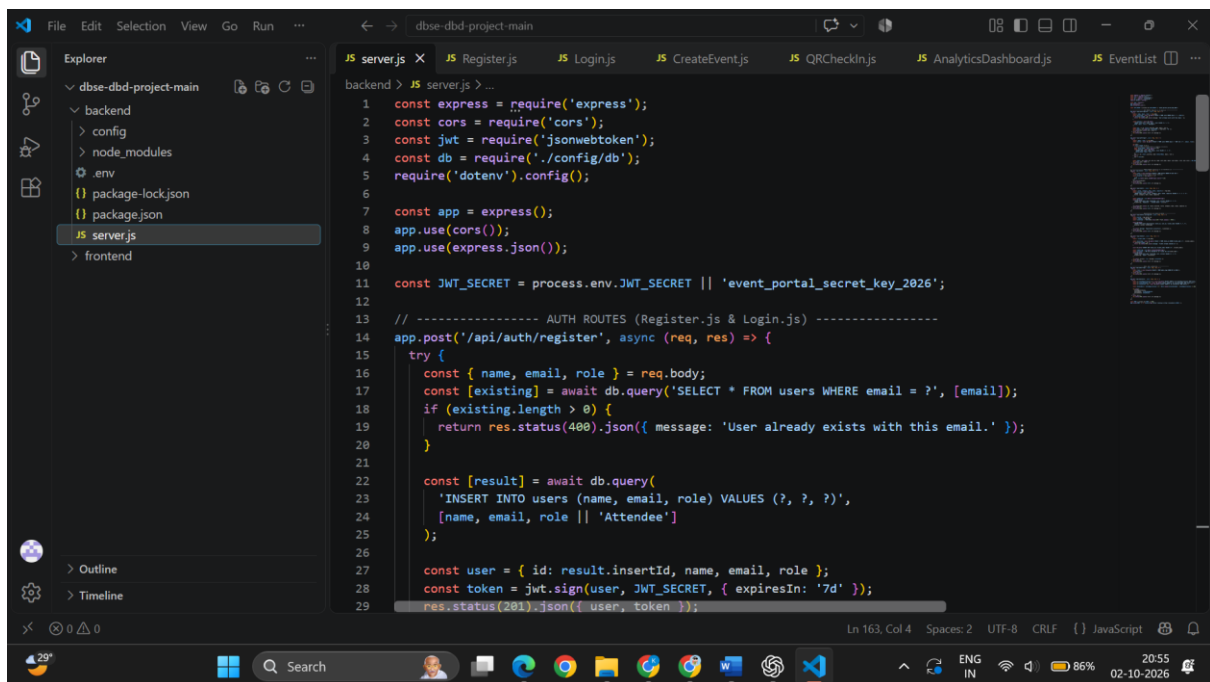


Sample code snippets



The screenshot shows the Visual Studio Code editor with the 'dbse-dbd-project-main' workspace. The Explorer sidebar on the left shows the project structure, with the 'App.js' file selected in the 'src' directory. The main editor displays the code for 'App.js', which is a React application. It imports various components and pages, initializes state for events and audit logs, and defines the App function.

```
1 import React, { useState } from 'react';
2 import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
3 import { AuthProvider } from './context/AuthContext';
4 import Navbar from './components/Navbar';
5 import ProtectedRoute from './components/ProtectedRoute';
6 import NotificationBanner from './components/NotificationBanner';
7
8 import Login from './pages/Login';
9 import Register from './pages/Register';
10 import EventList from './pages/EventList';
11 import CreateEvent from './pages/CreateEvent';
12 import QRCheckIn from './pages/QRCheckIn';
13 import AuditLogs from './pages/AuditLogs';
14 import AnalyticsDashboard from './pages/AnalyticsDashboard';
15
16 const initialEvents = [
17   { id: 1, title: 'Tech Conference 2026', category: 'Technology', date: '2026-10-15', venue: 'Hall A', capacity: 500 },
18   { id: 2, title: 'Annual Hackathon', category: 'Workshops', date: '2026-11-01', venue: 'Lab 3', capacity: 100 },
19   { id: 3, title: 'Leadership Summit', category: 'Management', date: '2026-12-05', venue: 'Auditorium', capacity: 200 }
20 ];
21
22 const initialLogs = [
23   { id: 1, timestamp: '10:15 AM', user: 'Organizer', action: 'Created event: Tech Conference 2026' },
24   { id: 2, timestamp: '10:30 AM', user: 'Staff', action: 'Verified QR Ticket Check-In #TCK-102' }
25 ];
26
27 export default function App() {
28   const [events, setEvents] = useState(initialEvents);
29   const [auditLogs, setAuditLogs] = useState(initialLogs);
```



The screenshot shows the Visual Studio Code editor with the 'dbse-dbd-project-main' workspace. The Explorer sidebar on the left shows the project structure, with the 'server.js' file selected in the 'backend' directory. The main editor displays the code for 'server.js', which is a Node.js application. It sets up an Express server, configures CORS, JWT, and database connections, and defines routes for user registration and login.

```
1 const express = require('express');
2 const cors = require('cors');
3 const jwt = require('jsonwebtoken');
4 const db = require('./config/db');
5 require('dotenv').config();
6
7 const app = express();
8 app.use(cors());
9 app.use(express.json());
10
11 const JWT_SECRET = process.env.JWT_SECRET || 'event_portal_secret_key_2026';
12
13 // ----- AUTH ROUTES (Register.js & Login.js) -----
14 app.post('/api/auth/register', async (req, res) => {
15   try {
16     const { name, email, role } = req.body;
17     const [existing] = await db.query('SELECT * FROM users WHERE email = ?', [email]);
18     if (existing.length > 0) {
19       return res.status(400).json({ message: 'User already exists with this email.' });
20     }
21
22     const [result] = await db.query(
23       'INSERT INTO users (name, email, role) VALUES (?, ?, ?)',
24       [name, email, role || 'Attendee']
25     );
26
27     const user = { id: result.insertId, name, email, role };
28     const token = jwt.sign(user, JWT_SECRET, { expiresIn: '7d' });
29     res.status(201).json({ user, token });
```

Installation/setup instructions

The **Event Management and Analytics Portal** is easy to set up because the submitted project is primarily developed using **React, HTML, CSS, and JavaScript**. The project can be installed and run locally using Node.js and npm. The setup can be completed in a few simple steps.

First, download or copy the complete **Event Management and Analytics Portal** project folder to the local computer. If the project is provided as a ZIP file, extract it using any standard extraction tool. Make sure that the complete project structure, including the package.json, src folder, and other required project files, is available inside the project directory.

Next, open the project folder using a development editor such as **Visual Studio Code**. Open the terminal inside the project folder and install the required project dependencies using the npm install command. This installs the libraries required by the application, including Axios, Chart.js, react-chartjs-2, and html5-qrcode.

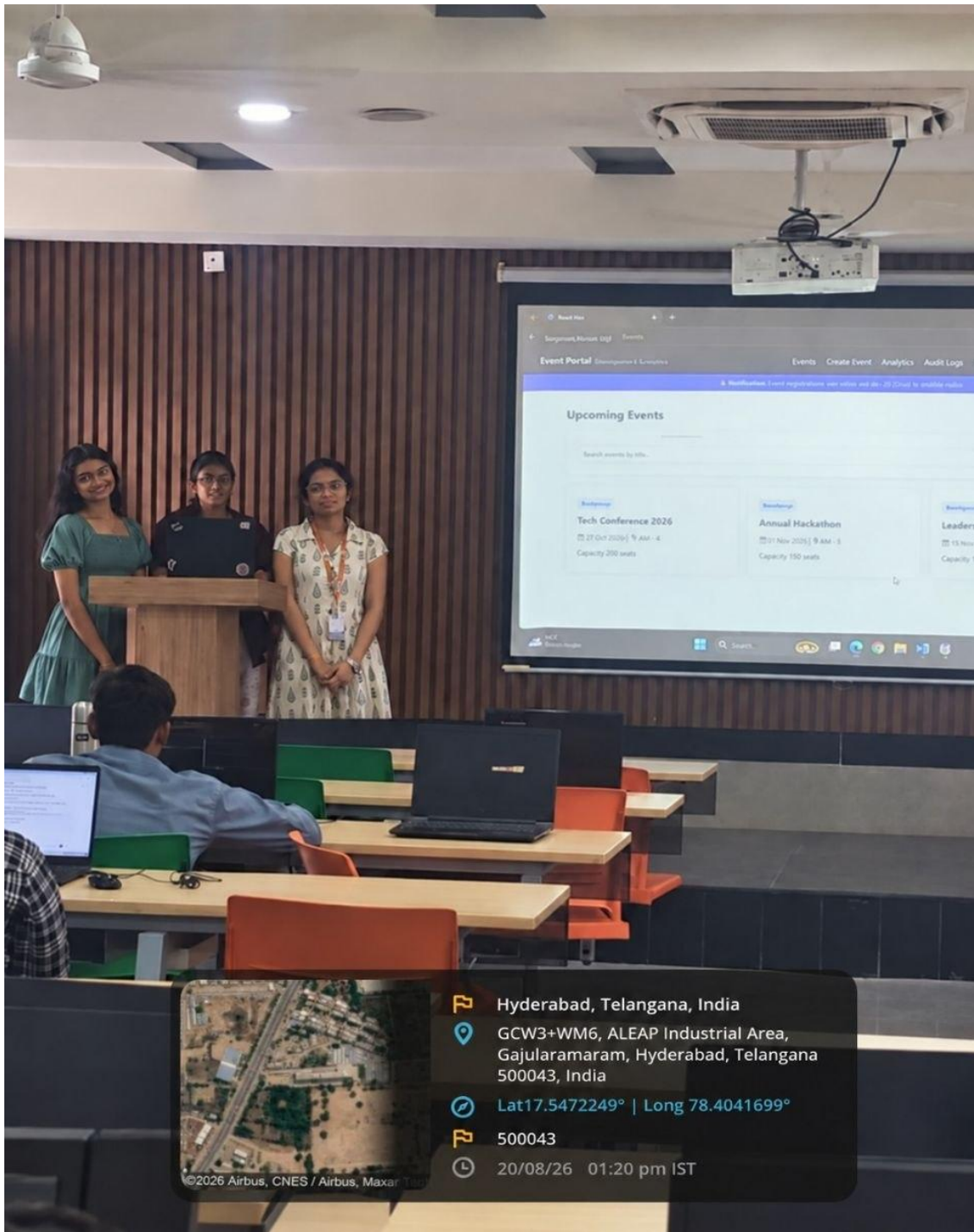
After installing the dependencies, run the application using the npm start command. The React development server will start the application, and the portal can be opened in a modern web browser such as **Google Chrome or Microsoft Edge**. The user can then access the different pages and test the available features of the system.

The application provides modules such as **Login, Registration, Event Listing, Event Creation, Search and Filtering, Event Registration, QR Check-In, Audit Logs, and Analytics Dashboard**. Users can navigate through these modules according to their assigned role. The QR Check-In feature may require permission to access the device camera when it is used for scanning.

The project also contains an Axios-based API integration layer. The API address can be configured using the REACT_APP_API_URL environment variable. If no API URL is configured, the application uses http://localhost:5000/api as the default address. The submitted project does not contain the complete Node.js/Express backend or MySQL database, so these components are planned for future integration.

Overall, the application can be easily prepared for classroom demonstration and development by installing Node.js, installing the project dependencies, and starting the React application. For production deployment, the project can be converted into an optimized build using npm run build and hosted on an appropriate web server or cloud platform.

Geo Tag photos with guide



Hyderabad, Telangana, India
GCW3+WM6, ALEAP Industrial Area,
Gajularamaram, Hyderabad, Telangana
500043, India
Lat17.5472249° | Long 78.4041699°
500043
20/08/26 01:20 pm IST

Review forms with guide signatures



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandamaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043

Phone No: 7815926876, www.klh.edu.in

B. Tech – CSE/CSIT/AI&DS **Rubrics and Evaluation Form**

A.Y 2025-26

Review 1: Rubrics and Evaluation

Date: 16-10-25

Course Code	24SDCS01		
Course Name	Frontend Development Frameworks		
Name of the Guide	Dr. Subbarayudu		
Group No: 17 Project Team	Roll No:	Name	Phone No
	2420080029	B. Manaswini	9347668993
	2420030178	S. Nitesh	
	2420030256	M. Sai Srujan	8686927073
Project Title	Design and development of an Integrated tourism platform for flights, trains, Hotels and hidden destinations.		

Review Comments from Guide or panel members:

1. Redefine problem statement
2. need proper PPT
- 3.
- 4.
- 5.

[Signature]

S.no	Review I- Rubrics	6	7	8	9	10	Marks
1	Survey Conducted	No work done	Survey conducted but partial	Survey conducted is satisfied	Extensive survey conducted through online	Extensive survey conducted including field survey	9
2	Survey Summary	No work done	Survey summary is partial	Survey summary is satisfied	Unique points identified from summary	Excellent feature or outcome is identified from survey summary as application yet to develop	10
3	Design Thinking Concepts (Empathy Mapping, Personas, Customer Journey Map, etc...)	No work done	Design thinking concepts are partial	Design thinking concepts are satisfied	Design thinking concepts are good	Design thinking concepts are excellent	10
4	Innovative and Unique Add-on to the Application	No work done	Innovative ideology is in partial state	Identified innovative / unique points for application	Good deviation from existing app	Excellent innovation	9
5	Module Identification (Sample user, admin, authentication, core functionality modules, result display module, feedback module, information module, etc...)	No work done	Modules are partial	Requirements are satisfied for the project	Requirements are identified in good level	All the requirements are completed	9
6	Article Publishing	No work done	Partially published	Article is satisfactory	Article is good	Article is excellent	9
7	Prototype Creation	No work done	Prototype is partial	Prototype is satisfactory	Prototype is good	Prototype is excellent	10
8	Youtube Video on Explaining the Prototype	No work done	Video prepared in offline and yet to upload	NA	NA	Video published	10
9	Team Coordination	No work done	Individual student's coordination with team is just ok	Individual student's coordination with team is satisfactory	Individual student's coordination with team is good	Individual student's coordination with team is excellent	9
10	Overall Project Progress	No work done	There is progress but not enough	Project progress is satisfactory	Project progress is good	Project progress is excellent	10
Total Avg Marks							10

Signature with Guide ID (8883)